

INDEX

Serial No.	Problem Statement	Page No.	Teacher's Signature
1	Write a program to implement a Diagonal Matrix, a Lower Triangular Matrix, an Upper Triangular Matrix, and a Symmetric Matrix using a one-dimensional array.	2	
2	Write a program to search an element from a list. Give user the option to perform Linear or Binary search.	9	
3	WAP to sort a list of elements. Give user the option to perform sorting using Insertion sort, Bubble sort or Selection sort.	15	
4	Implement Linked List. Include functions for insertion, deletion and search of a number, reverse the list and concatenate two linked lists (include a function and also overload operator +).	21	
5	Implement Doubly Linked List. Include functions for insertion, deletion and search of a number, reverse the list.	32	
6	Implement Circular Linked List. Include functions for insertion, deletion and search of a number, reverse the list.	41	
7	Perform Stack operations using Linked List implementation.	51	
8	Perform Stack operations using Array implementation.	56	
9	Perform Queue operations using Array.	60	
10	Perform Circular Queue operations using Array.	65	
11	Perform Queue operations using Linked List.	70	
12	Create and perform different operations on Double-ended Queues using Linked List implementation.	74	
13	WAP to scan a polynomial using linked list and add two polynomials.	80	
14	WAP to calculate factorial and to compute the factors of a given no. (i) using recursion, (ii) using iteration.	87	
15	WAP to display Fibonacci series (i) using recursion, (ii) using iteration.	91	
16	WAP to create a Binary Search Tree and include following operations in tree: i. Insertion (Recursive and Iterative Implementation) ii. Deletion by copying iii. Deletion by Merging iv. Search a no. in BST v. Display its preorder, postorder and inorder traversals Recursively vi. Display its preorder, postorder and inorder traversals Iteratively vii. Display its level-by-level traversals viii. Count the non-leaf nodes and leaf nodes ix. Display height of tree x. Create a mirror image of tree xi. Check whether two BSTs are equal or not	94	
17	WAP to convert the Sparse Matrix into non-zero form and vice-versa.	112	
18	WAP to reverse the order of the elements in the stack using additional stack.	118	
19	WAP to reverse the order of the elements in the stack using additional Queue.	122	

Assignment 1

Problem Statement:

Write a program to implement a Diagonal Matrix, a Lower Triangular Matrix, an Upper Triangular Matrix, and a Symmetric Matrix using a one-dimensional array.

Problem Definition:

Matrix memory optimization means storing special matrices efficiently by avoiding storage of predictable zero or duplicate elements, using compact 1D array instead of full 2D array.

- **Diagonal Matrix:** Only diagonal elements ($i == j$) are non-zero. Stored using the formula $arr[i]$.
- **Lower Triangular Matrix:** All elements above the main diagonal are zero. Non-zero elements mapped using $i * (i+1)/2 + j$.
- **Upper Triangular Matrix:** All elements below the main diagonal are zero. Non-zero elements mapped using $n * i - i * (i-1)/2 + (j-i)$.
- **Symmetric Matrix:** Matrix equal to its transpose ($A[i][j] = A[j][i]$). Only the lower triangular half is stored, using $i * (i+1)/2 + j$.

Algorithm:

Diagonal Matrix:

```
procedure setDiagonal(arr, i, j, x)
begin
    if(i == j)
        arr[i] <- x;
    end if
end procedure

procedure getDiagonal(arr, i, j)
begin
    if(i == j)
        return arr[i];
    else
        return 0;
    end if
end procedure
```

Lower Triangular Matrix:

```
procedure setLower(arr, i, j, x)
begin
    if(i >= j)
        arr[i*(i+1)/2+j] <- x;
    end if
end procedure
```

```

procedure getLower(arr, i, j)
begin
    if(i >= j)
        return arr[i*(i+1)/2+j];
    else
        return 0;
    end if
end procedure

```

Upper Triangular Matrix:

```

Procedure setUpper(arr, n, i, j, x)
begin
    if(i <= j)
        arr[n*i-i*(i-1)/2+(j-i)] <- x;
    end if
end procedure

Procedure getUpper(arr, n, i, j)
begin
    if(i <= j)
        return arr[n*i-i*(i-1)/2+(j-i)];
    else
        return 0;
    end if
end procedure

```

Symmetric Matrix:

```

Procedure setSymmetric(arr, i, j, x)
begin
    if(i >= j)
        arr[i*(i+1)/2+j] <- x;
    else
        arr[j*(j+1)/2+i] <- x;
    end if
end procedure

Procedure getSymmetric(arr, i, j)
begin
    if(i >= j)
        return arr[i*(i+1)/2+j];
    else
        return arr[j*(j+1)/2+i];
    end if
end procedure

```

Source Code:

```
#include <iostream>

using namespace std;

class Matrix
{
private:
    int n, *arr;

public:
    Matrix(int size)
    {
        n = size;
        arr = new int[n * (n + 1) / 2];
    }

    void setDiagonal(int i, int j, int x)
    {
        if (i == j)
            arr[i] = x;
    }

    void setLowerTri(int i, int j, int x)
    {
        if (i >= j)
            arr[i * (i + 1) / 2 + j] = x;
    }

    void setUpperTri(int i, int j, int x)
    {
        if (i <= j)
            arr[n * i - (i * (i - 1) / 2) + (j - i)] = x;
    }

    void setSymmetric(int i, int j, int x)
    {
        (i >= j) ? arr[i * (i + 1) / 2 + j] = x : arr[j * (j + 1) / 2 + i] = x;
    }

    int getDiagonal(int i, int j)
    {
        return (i == j) ? arr[i] : 0;
    }

    int getLowerTri(int i, int j)
    {
        return (i >= j) ? arr[i * (i + 1) / 2 + j] : 0;
    }

    int getUpperTri(int i, int j)
    {

```

```

        return (i <= j) ? arr[n * i - i * (i - 1) / 2 + (j - i)] : 0;
    }
    int getSymmetric(int i, int j)
    {
        return (i >= j) ? arr[i * (i + 1) / 2 + j] : arr[j * (j + 1) / 2 + i];
    }

    void display(int type)
    {
        cout << endl
            << "Matrix:" << endl;
        for (int i = 0; i < n; i++)
        {
            for (int j = 0; j < n; j++)
            {
                if (type == 1)
                    cout << getDiagonal(i, j) << " ";
                else if (type == 2)
                    cout << getLowerTri(i, j) << " ";
                else if (type == 3)
                    cout << getUpperTri(i, j) << " ";
                else if (type == 4)
                    cout << getSymmetric(i, j) << " ";
            }
            cout << endl;
        }
    }

    ~Matrix()
    {
        delete[] arr;
    }
};

int main()
{
    int n, choice, x;
    cout << "Enter the order of the matrix : ";
    cin >> n;
    Matrix M(n);

    cout << "\n1. Diagonal Matrix.";
    cout << "\n2. Lower Triangular Matrix.";
    cout << "\n3. Upper Triangular Matrix.";
    cout << "\n4. Symmetric Matrix.";
    cout << "\nEnter your choice : ";
    cin >> choice;

```

```

switch (choice)
{
case 1:
    cout << "\nEnter the diagonal elements : ";
    for (int i = 0; i < n; i++)
    {
        cin >> x;
        M.setDiagonal(i, i, x);
    }
    M.display(1);
    break;

case 2:
    cout << "\nEnter the elements : ";
    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j <= i; j++)
        {
            cin >> x;
            M.setLowerTri(i, j, x);
        }
    }
    M.display(2);
    break;

case 3:
    cout << "\nEnter the elements : ";
    for (int i = 0; i < n; i++)
    {
        for (int j = i; j < n; j++)
        {
            cin >> x;
            M.setUpperTri(i, j, x);
        }
    }
    M.display(3);
    break;

case 4:
    cout << "\nEnter the elements : ";
    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j <= i; j++)
        {
            cin >> x;
            M.setSymmetric(i, j, x);
        }
    }
}

```

```

        M.display(4);
        break;

default:
    cout << "Invalid Choice.";
}
return 0;
}

```

Output:

```

notamitgamer@fedora:~/Documents/bsc/semester_2/assignments$ ./a.out
Enter the order of the matrix : 3

1. Diagonal Matrix.
2. Lower Triangular Matrix.
3. Upper Triangular Matrix.
4. Symmetric Matrix.
Enter your choice : 1

Enter the diagonal elements : 1 2 3

Matrix:
1 0 0
0 2 0
0 0 3
notamitgamer@fedora:~/Documents/bsc/semester_2/assignments$ ./a.out
Enter the order of the matrix : 3

1. Diagonal Matrix.
2. Lower Triangular Matrix.
3. Upper Triangular Matrix.
4. Symmetric Matrix.
Enter your choice : 2

Enter the elements : 1 2 3 4 5 6

Matrix:
1 0 0
2 3 0
4 5 6
notamitgamer@fedora:~/Documents/bsc/semester_2/assignments$ ./a.out
Enter the order of the matrix : 3

1. Diagonal Matrix.
2. Lower Triangular Matrix.
3. Upper Triangular Matrix.
4. Symmetric Matrix.
Enter your choice : 3

Enter the elements : 1 2 3 4 5 6

Matrix:
1 2 3
0 4 5
0 0 6
notamitgamer@fedora:~/Documents/bsc/semester_2/assignments$ ./a.out
Enter the order of the matrix : 3

1. Diagonal Matrix.
2. Lower Triangular Matrix.
3. Upper Triangular Matrix.
4. Symmetric Matrix.
Enter your choice : 4

Enter the elements : 1 2 3 4 5 6

Matrix:
1 2 4
2 3 5
4 5 6

```

Discussion:

This program demonstrates how memory can be saved when working with special types of matrices — **Diagonal**, **Lower Triangular**, **Upper Triangular**, and **Symmetric** — by storing only their essential elements in a single dynamically allocated 1D array, rather than a full 2D array.

A **Matrix** class is used to bundle together the storage (the array) and the logic needed to interpret it correctly for each matrix type.

Two private members drive the class:

- **arr** - a dynamically allocated integer pointer that holds the compact array of stored values.
- **n** - the order (size) of the square matrix.

When a **Matrix** object is created, the constructor computes the array size as $n*(n+1)/2$ and allocates it on the heap using **new**.

The class's functionality is built around these key methods:

- **setDiagonal()**, **setLowerTri()**, **setUpperTri()** and **setSymmetric()** – Each of these accepts a row index, column index, and a value, then applies a matrix-specific formula to determine exactly where that value should be placed in the 1D array.
- **getDiagonal()**, **getLowerTri()**, **getUpperTri()** and **getSymmetric()** – These reconstruct values for any **(i, j)** position by locating them in the array (using ternary conditions), returning 0 for positions that were never stored, and — in the Symmetric case — reflecting values across the diagonal when needed.
- **display()** – Using two nested loops, this method walks through every position of the conceptual $n \times n$ matrix and prints the corresponding value, effectively rebuilding the full matrix view from the compact array.

Finally, the destructor **~Matrix()** ensures proper cleanup by releasing the dynamically allocated array with **delete[]** once the object goes out of scope, preventing memory leaks.

Assignment 2

Problem Statement:

Write a program to search an element from a list of elements using Linear Search and Binary Search techniques.

Problem Definition:

Searching is the process of finding whether a particular element is present in a given collection of elements, and if present, locating its position.

- **Linear Search:** Examines each element of the list sequentially, one by one, starting from the first, until the desired element is found or the list ends. It works on both sorted and unsorted lists but has a time complexity of $O(n)$.
- **Binary Search:** Works only on a sorted list. It repeatedly divides the search interval in half — comparing the key with the middle element and eliminating the half of the list where the key cannot exist — giving a time complexity of $O(\log n)$.

Algorithm:

Linear Search:

```
procedure linearSearch(arr, n, key)
begin
    for i ← 0 to n - 1 do
        if(arr[i] = key)
            return i;
        end if
    end for
    return -1;
end procedure
```

Binary Search:

```
procedure binarySearch(arr, n, key)
begin
    call insertionSort(arr, n);
    left ← 0;
    right ← n - 1;
    while(left ≤ right) do
        mid ← left + (right - left) / 2;
        if(arr[mid] = key)
            return mid;
        else if(arr[mid] < key)
            left ← mid + 1;
        else
            right ← mid - 1;
        end if
    end while
    return -1;
end procedure
```

Source Code:

```
#include <iostream>
#include <iomanip>
using namespace std;

typedef struct MenuItem {
    int id;
    const char* name;
} menu;

class SearchList {
private:
    int *arr;
    int n;

    void insertionSort() {
        for (int i = 1; i < n; i++) {
            int key = arr[i];
            int j = i - 1;
            while (j >= 0 && arr[j] > key) {
                arr[j + 1] = arr[j];
                j--;
            }
            arr[j + 1] = key;
        }
    }

public:
    SearchList(int size) {
        n = size;
        arr = new int[n];
    }

    void inputElements() {
        cout << "\nEnter " << n << " elements: ";
        for (int i = 0; i < n; i++) {
            cin >> arr[i];
        }
    }

    int linearSearch(int key) {
        for (int i = 0; i < n; i++) {
            if (arr[i] == key) {
                return i;
            }
        }
        return -1;
    }
};
```

```

}

int binarySearch(int key) {

    insertionSort();
    cout << "\n(Array automatically sorted for Binary Search)";
    display();

    int left = 0;
    int right = n - 1;

    while (left <= right) {
        int mid = left + (right - left) / 2;

        if (arr[mid] == key) {
            return mid;
        } else if (arr[mid] < key) {
            left = mid + 1;
        } else {
            right = mid - 1;
        }
    }
    return -1;
}

void display() {
    cout << "\nCurrent List: ";
    for (int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }
    cout << endl;
}

void printMenu() {
    menu items[] = {
        {1, "Linear Search"}, {2, "Binary Search"},
        {0, "Display List"}, {-1, "Exit"}
    };

    cout << "\n## Choose Search Method --\n-----"
    -\n";

    for (int i = 0; i < 4; i++) {
        cout << items[i].id << ". " << left << setw(20) << items[i].name;
        if (i % 2 == 1) cout << "\n";
    }
}

~SearchList() {

```

```

        delete[] arr;
    }
};

int main() {
    int n, choice, key;
    cout << "Enter the size of the list: ";
    cin >> n;

    if (n <= 0) {
        cout << "Invalid list size!\n";
        return 0;
    }

    SearchList list(n);
    list.inputElements();

    while (true) {
        list.printMenu();
        cout << "Enter your choice: ";
        cin >> choice;

        switch (choice) {
            case 1: {
                cout << "\nEnter element to search: ";
                cin >> key;
                int index = list.linearSearch(key);
                if (index != -1)
                    cout << "\nElement " << key << " found at index " << index << ".\n";
                else
                    cout << "\nElement " << key << " not found in the list.\n";
                break;
            }
            case 2: {
                cout << "\nEnter element to search: ";
                cin >> key;
                int index = list.binarySearch(key);
                if (index != -1)
                    cout << "\nElement " << key << " found at index " << index << " in
sorted array.\n";
                else
                    cout << "\nElement " << key << " not found in the list.\n";
                break;
            }
            case 0:
                list.display();
                break;
            case -1:

```

```

        cout << "\nExiting program.....\nMemory cleared!\nExit 0\n";
        return 0;
default:
        cout << "\nEntered wrong choice. Try again.\n";
    }
}
}

```

Output:

```

notamitgamer@fedora:~/bsc/semester_2/assignments on 7 main !?
~ > g++ assignment_01.cpp

notamitgamer@fedora:~/bsc/semester_2/assignments on 7 main !?
~ > ./a.out
Enter the size of the list: 5

Enter 5 elements: 2 1 5 3 9

## Choose Search Method --
-----
1. Linear Search      2. Binary Search
0. Display List      -1. Exit
Enter your choice: 1

Enter element to search: 3

Element 3 found at index 3.

## Choose Search Method --
-----
1. Linear Search      2. Binary Search
0. Display List      -1. Exit
Enter your choice: 1

Enter element to search: 8

Element 8 not found in the list.

## Choose Search Method --
-----
1. Linear Search      2. Binary Search
0. Display List      -1. Exit
Enter your choice: 2

Enter element to search: 5

(Array automatically sorted for Binary Search)
Current List: 1 2 3 5 9

Element 5 found at index 3 in sorted array.

## Choose Search Method --
-----
1. Linear Search      2. Binary Search
0. Display List      -1. Exit
Enter your choice: 0

Current List: 1 2 3 5 9

## Choose Search Method --
-----
1. Linear Search      2. Binary Search
0. Display List      -1. Exit
Enter your choice: -1

Exiting program.....
Memory cleared!
Exit 0

notamitgamer@fedora:~/bsc/semester_2/assignments on 7 main !?
~ >

```

Discussion:

This program demonstrates two fundamental searching techniques — **Linear Search** and **Binary Search** — wrapped inside a **SearchList** class that manages a dynamically allocated array of integers.

Two private members support the class:

- **arr** – a dynamically allocated integer pointer holding the list of elements.
- **n** – the total number of elements in the list.

A helper method, **insertionSort()**, is kept private since it is only meant to be used internally to prepare the array before a binary search is performed.

The class exposes the following key methods:

- **inputElements()** – Reads `n` integers from the user into the array.
- **linearSearch()** – Iterates through the array sequentially, comparing each element with the key, and returns the index where a match is found, or `-1` if the key doesn't exist in the list.
- **binarySearch()** – First sorts the array using **insertionSort()**, since binary search only works correctly on sorted data, then uses two pointers, **left** and **right**, to repeatedly narrow down the search range by comparing the key against the middle element until it is found or the range is exhausted.
- **display()** – Prints all current elements of the list.
- **printMenu()** – Uses a globally defined `MenuItem` struct (aliased as `menu`) to store menu options as `{id, name}` pairs in a local array, then loops through and prints them in a formatted two-column layout using `setw()`.

The **main()** function drives the program using an infinite loop with a `switch` statement, allowing the user to repeatedly choose between performing a linear search, a binary search, displaying the list, or exiting the program. Since binary search requires sorted data, the program automatically sorts and displays the array before performing that operation, keeping the underlying array consistent for future operations.

Finally, the destructor **~SearchList()** releases the dynamically allocated array using `delete[]` when the object is destroyed, avoiding memory leaks.

Assignment 3

Problem Statement:

Write a program to sort a list of elements. Give the user the option to perform sorting using Insertion Sort, Bubble Sort, or Selection Sort.

Problem Definition:

Sorting is the process of arranging elements of a list in a particular order (ascending/descending).

- **Insertion Sort:** Builds the sorted list one element at a time by picking an element and inserting it into its correct position among the already-sorted part. Time complexity $O(n^2)$.
- **Bubble Sort:** Repeatedly compares adjacent elements and swaps them if out of order, pushing the largest element to the end each pass. Time complexity $O(n^2)$.
- **Selection Sort:** Repeatedly finds the minimum element from the unsorted part and places it at the beginning. Time complexity $O(n^2)$.

Algorithm:

Insertion Sort:

```
procedure insertionSort(arr, n)
begin
    for i ← 1 to n - 1 do
        key ← arr[i];
        j ← i - 1;
        while(j >= 0 and arr[j] > key) do
            arr[j + 1] ← arr[j];
            j ← j - 1;
        end while
        arr[j + 1] ← key;
    end for
end procedure
```

Bubble Sort:

```
procedure bubbleSort(arr, n)
begin
    for i ← 0 to n - 2 do
        swapped ← false;
        for j ← 0 to n - i - 2 do
            if(arr[j] > arr[j + 1])
                swap(arr[j], arr[j + 1]);
                swapped ← true;
            end if
        end for
        if(swapped = false)
            break;
        end if
    end for
end procedure
```

```
                end if
            end for
        end procedure
```

Selection Sort:

```
procedure selectionSort(arr, n)
begin
    for i ← 0 to n - 2 do
        minIdx ← i;
        for j ← i + 1 to n - 1 do
            if(arr[j] < arr[minIdx])
                minIdx ← j;
            end if
        end for
        if(minIdx ≠ i)
            swap(arr[i], arr[minIdx]);
        end if
    end for
end procedure
```

Source Code:

```
#include <iostream>
#include <iomanip>
using namespace std;

typedef struct MenuItem {
    int id;
    const char* name;
} menu;

class SortList {
private:
    int *arr;
    int *tempArr;
    int n;

    void swapElements(int &a, int &b) {
        int temp = a;
        a = b;
        b = temp;
    }

    void reset() {
        for (int i = 0; i < n; i++) tempArr[i] = arr[i];
    }
}
```

```

public:
    SortList(int size) {
        n = size;
        arr = new int[n];
        tempArr = new int[n];
    }

    void inputElements() {
        cout << "\nEnter " << n << " elements: ";
        for (int i = 0; i < n; i++) {
            cin >> arr[i];
            tempArr[i] = arr[i];
        }
    }

    void insertionSort() {
        reset();
        for (int i = 1; i < n; i++) {
            int key = tempArr[i];
            int j = i - 1;
            while (j >= 0 && tempArr[j] > key) {
                tempArr[j + 1] = tempArr[j];
                j--;
            }
            tempArr[j + 1] = key;
        }
        display();
    }

    void bubbleSort() {
        reset();
        for (int i = 0; i < n - 1; i++) {
            bool swapped = false;
            for (int j = 0; j < n - i - 1; j++) {
                if (tempArr[j] > tempArr[j + 1]) {
                    swapElements(tempArr[j], tempArr[j + 1]);
                    swapped = true;
                }
            }
            if (!swapped) break;
        }
        display();
    }

    void selectionSort() {
        reset();
        for (int i = 0; i < n - 1; i++) {
            int minIdx = i;

```

```

        for (int j = i + 1; j < n; j++) {
            if (tempArr[j] < tempArr[minIdx]) minIdx = j;
        }
        if (minIdx != i) swapElements(tempArr[i], tempArr[minIdx]);
    }
    display();
}

void display() {
    cout << "Sorted Array: ";
    for (int i = 0; i < n; i++) cout << tempArr[i] << " ";
    cout << endl;
}

void displayOriginal() {
    cout << "\nOriginal List: ";
    for (int i = 0; i < n; i++) cout << arr[i] << " ";
    cout << endl;
}

void printMenu() {
    menu items[] = {
        {1, "Insertion Sort"}, {2, "Bubble Sort"},
        {3, "Selection Sort"}, {0, "Display Original"},
        {-1, "Exit"}
    };
    for (int i = 0; i < 5; i++) {
        cout << items[i].id << ". " << items[i].name << "\n";
    }
}

~SortList() {
    delete[] arr;
    delete[] tempArr;
}

};

int main() {
    int n, choice;
    cout << "Enter number of elements: ";
    cin >> n;
    if (n <= 0) {
        cout << "Invalid size!\n";
        return 0;
    }

    SortList sorter(n);
    sorter.inputElements();
}

```

```

while (true) {
    sorter.printMenu();
    cout << "Enter your choice: ";
    cin >> choice;

    switch (choice) {
        case 1: sorter.insertionSort(); break;
        case 2: sorter.bubbleSort(); break;
        case 3: sorter.selectionSort(); break;
        case 0: sorter.displayOriginal(); break;
        case -1:
            cout << "\nExiting program.....\nMemory cleared!\nExit 0\n";
            return 0;
        default:
            cout << "\nEntered wrong choice. Try again.\n";
    }
}
}

```

Output:

```

notamitgamer@fedora:~/bsc/semester_2/assignments on 7 main !?
~ > g++ assignment_02.cpp

notamitgamer@fedora:~/bsc/semester_2/assignments on 7 main !?
~ > ./a.out
Enter number of elements: 5

Enter 5 elements: 2 1 9 4 6

## Choose Sorting Method --
-----
1. Insertion Sort      2. Bubble Sort
3. Selection Sort      0. Display Original
-1. Exit
Enter your choice: 1

List sorted using Insertion Sort successfully!
Sorted Array: 1 2 4 6 9

## Choose Sorting Method --
-----
1. Insertion Sort      2. Bubble Sort
3. Selection Sort      0. Display Original
-1. Exit
Enter your choice: 2

List sorted using Bubble Sort successfully!
Sorted Array: 1 2 4 6 9

```

Continue to next page...

```

## Choose Sorting Method --
-----
1. Insertion Sort      2. Bubble Sort
3. Selection Sort      0. Display Original
-1. Exit
Enter your choice: 3

List sorted using Selection Sort successfully!
Sorted Array: 1 2 4 6 9

## Choose Sorting Method --
-----
1. Insertion Sort      2. Bubble Sort
3. Selection Sort      0. Display Original
-1. Exit
Enter your choice: 0

Original List: 2 1 9 4 6

## Choose Sorting Method --
-----
1. Insertion Sort      2. Bubble Sort
3. Selection Sort      0. Display Original
-1. Exit
Enter your choice: -1

Exiting program.....
Memory cleared!
Exit 0

notamitgamer@fedora:~/bsc/semester_2/assignments on 7 main !?
~ ) _

```

Discussion:

- **insertionSort()** copies `arr` into `tempArr` via `reset()`, then repeatedly takes an element (`key`) and shifts larger elements in the sorted left portion rightward until it finds the correct spot to insert the key.
- **bubbleSort()** makes repeated passes, swapping adjacent out-of-order elements; a `swapped` flag lets it exit early once a pass makes no swaps, avoiding unnecessary iterations on an already-sorted array.
- **selectionSort()** finds the index of the minimum element in the unsorted portion (`minIdx`) on each pass and swaps it into place at the front, growing the sorted portion by one each time.

All three operate on `tempArr`, a working copy of the original `arr`, so the original input remains unaffected and can be displayed separately via `displayOriginal()`.

Assignment 4

Problem Statement:

Implement a Singly Linked List. Include functions for insertion, deletion and search of a number, reverse the list, and concatenate two linked lists (include a function and also overload the `+` operator).

Problem Definition:

A **Linked List** is a linear data structure where elements (nodes) are connected using pointers, with each node storing data and a reference to the next node.

- **Insertion/Deletion:** Adding or removing a node at the beginning, end, or a given position by adjusting pointer links.
- **Search:** Traversing the list sequentially to find the position of a given value.
- **Reverse:** Reversing the direction of the `next` pointers so the list traverses in the opposite order.
- **Concatenation:** Joining two lists, done both via a function and an overloaded `+` operator. The function links the last node of the first list to the head of the second (modifying the first list), while `operator+` builds a brand-new list by copying the nodes of both lists.

Algorithm:

Insertion at Beginning:

```
procedure insertAtBegin(head, val)
begin
    newNode ← create new node;
    newNode.data ← val;
    newNode.next ← head;
    head ← newNode;
end procedure
```

Insertion at End:

```
procedure insertAtEnd(head, val)
begin
    newNode ← create new node;
    newNode.data ← val;
    newNode.next ← NULL;
    if(head = NULL)
        head ← newNode;
    else
        temp ← head;
        while(temp.next ≠ NULL) do
            temp ← temp.next;
        end while
        temp.next ← newNode;
    end if
end procedure
```

Insertion at Position:

```
procedure insertAtPos(head, pos, val)
begin
    if(pos < 1)
        print "Invalid Position";
        return;
    end if
    if(pos = 1)
        call insertAtBegin(head, val);
        return;
    end if
    temp ← head;
    for i ← 1 to pos - 2 do
        if(temp = NULL)
            break;
        end if
        temp ← temp.next;
    end for
    if(temp = NULL)
        print "Out of range";
        return;
    end if
    newNode ← create new node;
    newNode.data ← val;
    newNode.next ← temp.next;
    temp.next ← newNode;
end procedure
```

Deletion at Beginning:

```
procedure deleteAtBegin(head)
begin
    if(head = NULL)
        print "List is empty";
        return;
    end if
    temp ← head;
    head ← head.next;
    free temp;
end procedure
```

Deletion at End:

```
procedure deleteAtEnd(head)
begin
    if(head = NULL)
        print "List is empty";
        return;
```

```

        end if
        if(head.next = NULL)
            free head;
            head ← NULL;
            return;
        end if
        temp ← head;
        while(temp.next.next ≠ NULL) do
            temp ← temp.next;
        end while
        free temp.next;
        temp.next ← NULL;
    end procedure

```

Deletion at Position:

```

procedure deleteAtPos(head, pos)
begin
    if(head = NULL or pos < 1)
        print "Unable to process";
        return;
    end if
    if(pos = 1)
        call deleteAtBegin(head);
        return;
    end if
    temp ← head;
    for i ← 1 to pos - 2 do
        if(temp = NULL)
            break;
        end if
        temp ← temp.next;
    end for
    if(temp = NULL or temp.next = NULL)
        print "Out of range";
        return;
    end if
    delNode ← temp.next;
    temp.next ← delNode.next;
    free delNode;
end procedure

```

Search:

```

procedure search(head, key)
begin
    temp ← head;
    pos ← 1;

```

```

        while(temp ≠ NULL) do
            if(temp.data = key)
                return pos;
            end if
            temp ← temp.next;
            pos ← pos + 1;
        end while
        return -1;
    end procedure

```

Reverse:

```

procedure reverse(head)
begin
    prev ← NULL;
    curr ← head;
    nxt ← NULL;
    while(curr ≠ NULL) do
        nxt ← curr.next;
        curr.next ← prev;
        prev ← curr;
        curr ← nxt;
    end while
    head ← prev;
end procedure

```

Concatenate:

```

procedure concatenate(head1, head2)
begin
    if(head1 = NULL)
        head1 ← head2;
        head2 ← NULL;
        return;
    end if
    temp ← head1;
    while(temp.next ≠ NULL) do
        temp ← temp.next;
    end while
    temp.next ← head2;
    head2 ← NULL;
end procedure

```

Operator + (Concatenate into a New List):

```

procedure operator+(list1, list2)
begin
    result ← empty list;

```

```

    temp ← list1.head;
    while(temp ≠ NULL) do
        call insertAtEnd(result, temp.data);
        temp ← temp.next;
    end while
    temp ← list2.head;
    while(temp ≠ NULL) do
        call insertAtEnd(result, temp.data);
        temp ← temp.next;
    end while
    return result;
end procedure

```

Source Code:

```

#include <iostream>
using namespace std;

typedef struct nodeType {
    int data;
    nodeType *next = nullptr;
} node;

class SinglyLinkedList {
private:
    node *head;

public:
    SinglyLinkedList() { head = nullptr; }

    SinglyLinkedList(const SinglyLinkedList &other) {
        head = nullptr;
        node *temp = other.head;
        while (temp != nullptr) {
            insertAtEnd(temp->data);
            temp = temp->next;
        }
    }

    void insertAtBegin(int val) {
        node *newNode = new node;
        newNode->data = val;
        newNode->next = head;
        head = newNode;
    }

    void insertAtEnd(int val) {
        node *newNode = new node;

```

```

        newNode->data = val;
        newNode->next = nullptr;
        if (head == nullptr) {
            head = newNode;
        } else {
            node *temp = head;
            while (temp->next != nullptr) temp = temp->next;
            temp->next = newNode;
        }
    }

    void insertAtPos(int pos, int val) {
        if (pos < 1) { cout << "Invalid Position!\n"; return; }
        if (pos == 1) { insertAtBegin(val); return; }
        node *temp = head;
        for (int i = 1; i < pos - 1 && temp != nullptr; i++) temp = temp->next;
        if (temp == nullptr) { cout << "Out of range!\n"; return; }
        node *newNode = new node;
        newNode->data = val;
        newNode->next = temp->next;
        temp->next = newNode;
    }

    void deleteAtBegin() {
        if (head == nullptr) { cout << "List is empty!\n"; return; }
        node *temp = head;
        head = head->next;
        delete temp;
    }

    void deleteAtEnd() {
        if (head == nullptr) { cout << "List is empty!\n"; return; }
        if (head->next == nullptr) { delete head; head = nullptr; return; }
        node *temp = head;
        while (temp->next->next != nullptr) temp = temp->next;
        delete temp->next;
        temp->next = nullptr;
    }

    void deleteAtPos(int pos) {
        if (head == nullptr || pos < 1) { cout << "Unable to process!\n"; return; }
        if (pos == 1) { deleteAtBegin(); return; }
        node *temp = head;
        for (int i = 1; i < pos - 1 && temp != nullptr; i++) temp = temp->next;
        if (temp == nullptr || temp->next == nullptr) { cout << "Out of range!\n"; return; }

        node *delNode = temp->next;
        temp->next = delNode->next;
    }

```

```

        delete delNode;
    }

int search(int key) {
    node *temp = head;
    int pos = 1;
    while (temp != nullptr) {
        if (temp->data == key) return pos;
        temp = temp->next;
        pos++;
    }
    return -1;
}

void reverse() {
    node *prev = nullptr, *curr = head, *nxt = nullptr;
    while (curr != nullptr) {
        nxt = curr->next;
        curr->next = prev;
        prev = curr;
        curr = nxt;
    }
    head = prev;
}

void concatenate(SinglyLinkedList &other) {
    if (head == nullptr) { head = other.head; other.head = nullptr; return; }
    node *temp = head;
    while (temp->next != nullptr) temp = temp->next;
    temp->next = other.head;
    other.head = nullptr;
}

SinglyLinkedList operator+(const SinglyLinkedList &other) const {
    SinglyLinkedList result;
    node *temp = head;
    while (temp != nullptr) { result.insertAtEnd(temp->data); temp = temp->next; }
    temp = other.head;
    while (temp != nullptr) { result.insertAtEnd(temp->data); temp = temp->next; }
    return result;
}

void display() const {
    node *temp = head;
    cout << "List: ";
    while (temp != nullptr) { cout << temp->data << " -> "; temp = temp->next; }
    cout << "nullptr\n";
}

```

```

void printMenu() {
    cout << "1.insertAtBegin 2.insertAtEnd 3.insertAtPos 4.deleteAtBegin 5.deleteAtEnd
\n"
        << "6.deleteAtPos 7.search 8.reverse 9.concatenate 10.operator+ 0.Display
-1.Exit\n";
}

~SinglyLinkedList() {
    node *temp;
    while (head != nullptr) { temp = head; head = head->next; delete temp; }
}

};

int main() {
    SinglyLinkedList list1;
    int choice, val, pos;

    while (true) {
        list1.printMenu();
        cout << "Enter your choice: ";
        cin >> choice;
        switch (choice) {
            case 1: cout << "Enter value: "; cin >> val; list1.insertAtBegin(val); break;
            case 2: cout << "Enter value: "; cin >> val; list1.insertAtEnd(val); break;
            case 3:
                cout << "Enter position: "; cin >> pos;
                cout << "Enter value: "; cin >> val;
                list1.insertAtPos(pos, val);
                break;
            case 4: list1.deleteAtBegin(); break;
            case 5: list1.deleteAtEnd(); break;
            case 6: cout << "Enter position: "; cin >> pos; list1.deleteAtPos(pos); break;
            case 7: {
                cout << "Enter value to search: "; cin >> val;
                int res = list1.search(val);
                if (res != -1) cout << "Found at position " << res << ".\n";
                else cout << "Not found.\n";
                break;
            }
            case 8: list1.reverse(); break;
            case 9: {
                SinglyLinkedList list2;
                int m, tempVal;
                cout << "Enter number of elements in second list: "; cin >> m;
                for (int i = 0; i < m; i++) { cin >> tempVal; list2.insertAtEnd(tempVal);

                    list1.concatenate(list2);
                }
            }
        }
    }
}

```

```

        break;
    }
    case 10: {
        SinglyLinkedList list2;
        int m, tempVal;
        cout << "Enter number of elements in second list: "; cin >> m;
        for (int i = 0; i < m; i++) { cin >> tempVal; list2.insertAtEnd(tempVal);
    }

        SinglyLinkedList combined = list1 + list2;
        combined.display();
        break;
    }
    case 0: list1.display(); break;
    case -1: cout << "Exiting...\n"; return 0;
    default: cout << "Wrong choice. Try again.\n";
}
}
}

```

Output:

```
notamitgamer@fedora:~/bsc/semester_2/assignments on 7 main !?  
~ ) g++ assignment_03.cpp  
  
notamitgamer@fedora:~/bsc/semester_2/assignments on 7 main !?  
~ ) ./a.out  
1.insertAtBegin 2.insertAtEnd 3.insertAtPos 4.deleteAtBegin 5.deleteAtEnd  
6.deleteAtPos 7.search 8.reverse 9.concatenate 10.operator+ 0.Display -1.Exit  
Enter your choice: 1  
Enter value: 10  
1.insertAtBegin 2.insertAtEnd 3.insertAtPos 4.deleteAtBegin 5.deleteAtEnd  
6.deleteAtPos 7.search 8.reverse 9.concatenate 10.operator+ 0.Display -1.Exit  
Enter your choice: 2  
Enter value: 30  
1.insertAtBegin 2.insertAtEnd 3.insertAtPos 4.deleteAtBegin 5.deleteAtEnd  
6.deleteAtPos 7.search 8.reverse 9.concatenate 10.operator+ 0.Display -1.Exit  
Enter your choice: 3  
Enter position: 2  
Enter value: 20  
1.insertAtBegin 2.insertAtEnd 3.insertAtPos 4.deleteAtBegin 5.deleteAtEnd  
6.deleteAtPos 7.search 8.reverse 9.concatenate 10.operator+ 0.Display -1.Exit  
Enter your choice: 0  
  
List: 10 -> 20 -> 30 -> nullptr  
  
1.insertAtBegin 2.insertAtEnd 3.insertAtPos 4.deleteAtBegin 5.deleteAtEnd  
6.deleteAtPos 7.search 8.reverse 9.concatenate 10.operator+ 0.Display -1.Exit  
Enter your choice: 7  
Enter value to search: 20  
Found at position 2.  
1.insertAtBegin 2.insertAtEnd 3.insertAtPos 4.deleteAtBegin 5.deleteAtEnd  
6.deleteAtPos 7.search 8.reverse 9.concatenate 10.operator+ 0.Display -1.Exit  
Enter your choice: 7  
Enter value to search: 99  
Not found.  
1.insertAtBegin 2.insertAtEnd 3.insertAtPos 4.deleteAtBegin 5.deleteAtEnd  
6.deleteAtPos 7.search 8.reverse 9.concatenate 10.operator+ 0.Display -1.Exit  
Enter your choice: 8  
1.insertAtBegin 2.insertAtEnd 3.insertAtPos 4.deleteAtBegin 5.deleteAtEnd  
6.deleteAtPos 7.search 8.reverse 9.concatenate 10.operator+ 0.Display -1.Exit  
Enter your choice: 0  
  
List: 30 -> 20 -> 10 -> nullptr  
  
1.insertAtBegin 2.insertAtEnd 3.insertAtPos 4.deleteAtBegin 5.deleteAtEnd  
6.deleteAtPos 7.search 8.reverse 9.concatenate 10.operator+ 0.Display -1.Exit  
Enter your choice: 9  
Enter number of elements in second list: 2  
40  
50  
1.insertAtBegin 2.insertAtEnd 3.insertAtPos 4.deleteAtBegin 5.deleteAtEnd  
6.deleteAtPos 7.search 8.reverse 9.concatenate 10.operator+ 0.Display -1.Exit  
Enter your choice: 0
```

Continue to next page...

```

List: 30 -> 20 -> 10 -> 40 -> 50 -> nullptr

1.insertAtBegin 2.insertAtEnd 3.insertAtPos 4.deleteAtBegin 5.deleteAtEnd
6.deleteAtPos 7.search 8.reverse 9.concatenate 10.operator+ 0.Display -1.Exit
Enter your choice: 10
Enter number of elements in second list: 2
60
70

List: 30 -> 20 -> 10 -> 40 -> 50 -> 60 -> 70 -> nullptr

1.insertAtBegin 2.insertAtEnd 3.insertAtPos 4.deleteAtBegin 5.deleteAtEnd
6.deleteAtPos 7.search 8.reverse 9.concatenate 10.operator+ 0.Display -1.Exit
Enter your choice: 4
1.insertAtBegin 2.insertAtEnd 3.insertAtPos 4.deleteAtBegin 5.deleteAtEnd
6.deleteAtPos 7.search 8.reverse 9.concatenate 10.operator+ 0.Display -1.Exit
Enter your choice: 5
1.insertAtBegin 2.insertAtEnd 3.insertAtPos 4.deleteAtBegin 5.deleteAtEnd
6.deleteAtPos 7.search 8.reverse 9.concatenate 10.operator+ 0.Display -1.Exit
Enter your choice: 6
Enter position: 3
1.insertAtBegin 2.insertAtEnd 3.insertAtPos 4.deleteAtBegin 5.deleteAtEnd
6.deleteAtPos 7.search 8.reverse 9.concatenate 10.operator+ 0.Display -1.Exit
Enter your choice: 0

List: 20 -> 10 -> nullptr

1.insertAtBegin 2.insertAtEnd 3.insertAtPos 4.deleteAtBegin 5.deleteAtEnd
6.deleteAtPos 7.search 8.reverse 9.concatenate 10.operator+ 0.Display -1.Exit
Enter your choice: -1
Exiting...

notamitgamer@fedora:~/bsc/semester_2/assignments on ? main !?
~ )

```

Discussion:

- **insertAtBegin/insertAtEnd/insertAtPos** create a new node and link it into the chain — at the head, by traversing to the last node, or by traversing to the node just before the target position.
- **deleteAtBegin/deleteAtEnd/deleteAtPos** unlink a node by redirecting the previous node's **next** pointer around it before freeing it, with edge cases handled for an empty list or a single-node list.
- **search()** traverses the list sequentially, comparing each node's data with the key, returning its 1-based position or **-1**.
- **reverse()** rebuilds the list by walking through it once, flipping each node's **next** pointer to point backward using three tracking pointers (**prev** , **curr** , **nxt**).
- **concatenate()** links the last node of the calling list directly to the head of the other list, modifying the calling list in place.
- **operator+** builds an entirely new list by copying all nodes from both lists in sequence, leaving both original lists unmodified.

Assignment 5

Problem Statement:

Implement a Doubly Linked List. Include functions for insertion, deletion and search of a number, and reverse the list.

Problem Definition:

A **Doubly Linked List** is a linear data structure where each node stores data along with two pointers — `prev` and `next` — pointing to the previous and next nodes, allowing traversal in both directions.

- **Insertion/Deletion:** Adding or removing a node at the beginning, end, or a given position by updating both `prev` and `next` links.
- **Search:** Traversing the list sequentially to find the position of a given value.
- **Reverse:** Swapping the `prev` and `next` pointers of every node so the list's direction is flipped.

Algorithm:

Insertion at Beginning:

```
procedure insAtBegin(head, val)
begin
    newNode ← create new node;
    newNode.data ← val;
    newNode.prev ← NULL;
    newNode.next ← head;
    if(head ≠ NULL)
        head.prev ← newNode;
    end if
    head ← newNode;
end procedure
```

Insertion at End:

```
procedure insAtEnd(head, val)
begin
    newNode ← create new node;
    newNode.data ← val;
    newNode.next ← NULL;
    if(head = NULL)
        newNode.prev ← NULL;
        head ← newNode;
    else
        temp ← head;
        while(temp.next ≠ NULL) do
            temp ← temp.next;
        end while
        temp.next ← newNode;
        newNode.prev ← temp;
    end if
end procedure
```

```
        end if
    end procedure
```

Insertion at Position:

```
procedure insAtPos(head, pos, val)
begin
    if(pos < 1)
        print "Invalid Position";
        return;
    end if
    if(pos = 1)
        call insAtBegin(head, val);
        return;
    end if
    temp ← head;
    for i ← 1 to pos - 2 do
        if(temp = NULL)
            break;
        end if
        temp ← temp.next;
    end for
    if(temp = NULL)
        print "Out of range";
        return;
    end if
    newNode ← create new node;
    newNode.data ← val;
    newNode.next ← temp.next;
    newNode.prev ← temp;
    if(temp.next ≠ NULL)
        temp.next.prev ← newNode;
    end if
    temp.next ← newNode;
end procedure
```

Deletion at Beginning:

```
procedure delAtBegin(head)
begin
    if(head = NULL)
        print "List is empty";
        return;
    end if
    temp ← head;
    head ← head.next;
    if(head ≠ NULL)
        head.prev ← NULL;
    end if
end procedure
```

```
        end if
        free temp;
    end procedure
```

Deletion at End:

```
procedure delAtEnd(head)
begin
    if(head = NULL)
        print "List is empty";
        return;
    end if
    if(head.next = NULL)
        free head;
        head ← NULL;
        return;
    end if
    temp ← head;
    while(temp.next ≠ NULL) do
        temp ← temp.next;
    end while
    temp.prev.next ← NULL;
    free temp;
end procedure
```

Deletion at Position:

```
procedure delAtPos(head, pos)
begin
    if(head = NULL or pos < 1)
        print "Unable to process";
        return;
    end if
    if(pos = 1)
        call delAtBegin(head);
        return;
    end if
    temp ← head;
    for i ← 1 to pos - 1 do
        if(temp = NULL)
            break;
        end if
        temp ← temp.next;
    end for
    if(temp = NULL)
        print "Out of range";
        return;
    end if
```

```

        if(temp.next ≠ NULL)
            temp.next.prev ← temp.prev;
        end if
        if(temp.prev ≠ NULL)
            temp.prev.next ← temp.next;
        end if
        free temp;
    end procedure

```

Search:

```

procedure search(head, key)
begin
    if(head = NULL)
        print "List is empty";
        return;
    end if
    temp ← head;
    pos ← 1;
    while(temp ≠ NULL) do
        if(temp.data = key)
            print "Found at position: ", pos;
            return;
        end if
        temp ← temp.next;
        pos ← pos + 1;
    end while
    print "Element not found";
end procedure

```

Reverse:

```

procedure reverse(head)
begin
    if(head = NULL or head.next = NULL)
        return;
    end if
    curr ← head;
    temp ← NULL;
    while(curr ≠ NULL) do
        temp ← curr.prev;
        curr.prev ← curr.next;
        curr.next ← temp;
        curr ← curr.prev;
    end while
    if(temp ≠ NULL)
        head ← temp.prev;
    end if

```

end procedure

Source Code:

```
#include <iostream>
using namespace std;

typedef struct nodeType {
    int data;
    nodeType *prev = nullptr;
    nodeType *next = nullptr;
} node;

class DoublyLinkedList {
private:
    node *head;

public:
    DoublyLinkedList() { head = nullptr; }

    void insAtBegin() {
        node *newNode = new node;
        cout << "Enter the value: ";
        cin >> newNode->data;
        newNode->prev = nullptr;
        newNode->next = head;
        if (head != nullptr) head->prev = newNode;
        head = newNode;
    }

    void insAtEnd() {
        node *newNode = new node;
        cout << "Enter the value: ";
        cin >> newNode->data;
        newNode->next = nullptr;
        if (head == nullptr) {
            newNode->prev = nullptr;
            head = newNode;
        } else {
            node *temp = head;
            while (temp->next != nullptr) temp = temp->next;
            temp->next = newNode;
            newNode->prev = temp;
        }
    }

    void insAtPos() {
        int pos;
```

```

        cout << "Enter the position: ";
        cin >> pos;
        if (pos < 1) { cout << "Invalid Position!\n"; return; }
        if (pos == 1) { insAtBegin(); return; }
        node *temp = head;
        for (int i = 1; i < pos - 1 && temp != nullptr; i++) temp = temp->next;
        if (temp == nullptr) { cout << "Out of range!\n"; return; }
        node *newNode = new node;
        cout << "Enter the value: ";
        cin >> newNode->data;
        newNode->next = temp->next;
        newNode->prev = temp;
        if (temp->next != nullptr) temp->next->prev = newNode;
        temp->next = newNode;
    }

    void delAtBegin() {
        if (head == nullptr) { cout << "List is empty!\n"; return; }
        node *temp = head;
        head = head->next;
        if (head != nullptr) head->prev = nullptr;
        delete temp;
    }

    void delAtEnd() {
        if (head == nullptr) { cout << "List is empty!\n"; return; }
        if (head->next == nullptr) { delete head; head = nullptr; return; }
        node *temp = head;
        while (temp->next != nullptr) temp = temp->next;
        temp->prev->next = nullptr;
        delete temp;
    }

    void delAtPos() {
        if (head == nullptr) { cout << "List is empty!\n"; return; }
        int pos;
        cout << "Enter the position: ";
        cin >> pos;
        if (pos < 1) { cout << "Invalid Position!\n"; return; }
        if (pos == 1) { delAtBegin(); return; }
        node *temp = head;
        for (int i = 1; i < pos && temp != nullptr; i++) temp = temp->next;
        if (temp == nullptr) { cout << "Out of range!\n"; return; }
        if (temp->next != nullptr) temp->next->prev = temp->prev;
        if (temp->prev != nullptr) temp->prev->next = temp->next;
        delete temp;
    }
}

```

```

void search() {
    if (head == nullptr) { cout << "List is empty!\n"; return; }
    int key, pos = 1;
    cout << "Enter the element to search: ";
    cin >> key;
    node *temp = head;
    while (temp != nullptr) {
        if (temp->data == key) { cout << "Found at position: " << pos << "\n"; return; }
        temp = temp->next;
        pos++;
    }
    cout << "Element not found.\n";
}

void reverse() {
    if (head == nullptr || head->next == nullptr) return;
    node *curr = head, *temp = nullptr;
    while (curr != nullptr) {
        temp = curr->prev;
        curr->prev = curr->next;
        curr->next = temp;
        curr = curr->prev;
    }
    if (temp != nullptr) head = temp->prev;
}

void display() {
    node *temp = head;
    cout << "List: nullptr ";
    while (temp != nullptr) { cout << temp->data << " "; temp = temp->next; }
    cout << "nullptr\n";
}

void printMenu() {
    cout << "1.insertAtBegin 2.insertAtEnd 3.insertAtPos 4.deleteAtBegin 5.deleteAtEnd\n"
    << "6.deleteAtPos 7.search 8.reverse 0.Display -1.Exit\n";
}

~DoublyLinkedList() {
    node *temp;
    while (head != nullptr) { temp = head; head = head->next; delete temp; }
};

int main() {
    DoublyLinkedList list;

```

```

int choice;

while (true) {
    list.printMenu();
    cout << "Enter your choice: ";
    cin >> choice;

    switch (choice) {
        case 1: list.insAtBegin(); break;
        case 2: list.insAtEnd(); break;
        case 3: list.insAtPos(); break;
        case 4: list.delAtBegin(); break;
        case 5: list.delAtEnd(); break;
        case 6: list.delAtPos(); break;
        case 7: list.search(); break;
        case 8: list.reverse(); break;
        case 0: list.display(); break;
        case -1: cout << "Exiting...\n"; return 0;
        default: cout << "Wrong choice. Try again.\n";
    }
}
}

```

Output:

```
notamitgamer@fedora:~/bsc/semester_2/assignments on 7 main !?  
~ > g++ assignment_04.cpp  
  
notamitgamer@fedora:~/bsc/semester_2/assignments on 7 main !?  
~ > ./a.out  
1.insertAtBegin 2.insertAtEnd 3.insertAtPos 4.deleteAtBegin 5.deleteAtEnd  
6.deleteAtPos 7.search 8.reverse 0.Display -1.Exit  
Enter your choice: 1  
Enter the value: 10  
1.insertAtBegin 2.insertAtEnd 3.insertAtPos 4.deleteAtBegin 5.deleteAtEnd  
6.deleteAtPos 7.search 8.reverse 0.Display -1.Exit  
Enter your choice: 2  
Enter the value: 30  
1.insertAtBegin 2.insertAtEnd 3.insertAtPos 4.deleteAtBegin 5.deleteAtEnd  
6.deleteAtPos 7.search 8.reverse 0.Display -1.Exit  
Enter your choice: 3  
Enter the position: 2  
Enter the value: 20  
1.insertAtBegin 2.insertAtEnd 3.insertAtPos 4.deleteAtBegin 5.deleteAtEnd  
6.deleteAtPos 7.search 8.reverse 0.Display -1.Exit  
Enter your choice: 0  
List: nullptr 10 20 30 nullptr  
1.insertAtBegin 2.insertAtEnd 3.insertAtPos 4.deleteAtBegin 5.deleteAtEnd  
6.deleteAtPos 7.search 8.reverse 0.Display -1.Exit  
Enter your choice: 7  
Enter the element to search: 30  
Found at position: 3  
1.insertAtBegin 2.insertAtEnd 3.insertAtPos 4.deleteAtBegin 5.deleteAtEnd  
6.deleteAtPos 7.search 8.reverse 0.Display -1.Exit  
Enter your choice: 8  
1.insertAtBegin 2.insertAtEnd 3.insertAtPos 4.deleteAtBegin 5.deleteAtEnd  
6.deleteAtPos 7.search 8.reverse 0.Display -1.Exit  
Enter your choice: 0  
List: nullptr 30 20 10 nullptr  
1.insertAtBegin 2.insertAtEnd 3.insertAtPos 4.deleteAtBegin 5.deleteAtEnd  
6.deleteAtPos 7.search 8.reverse 0.Display -1.Exit  
Enter your choice: 4  
1.insertAtBegin 2.insertAtEnd 3.insertAtPos 4.deleteAtBegin 5.deleteAtEnd  
6.deleteAtPos 7.search 8.reverse 0.Display -1.Exit  
Enter your choice: 5  
1.insertAtBegin 2.insertAtEnd 3.insertAtPos 4.deleteAtBegin 5.deleteAtEnd  
6.deleteAtPos 7.search 8.reverse 0.Display -1.Exit  
Enter your choice: 6  
Enter the position: 1  
1.insertAtBegin 2.insertAtEnd 3.insertAtPos 4.deleteAtBegin 5.deleteAtEnd  
6.deleteAtPos 7.search 8.reverse 0.Display -1.Exit  
Enter your choice: 0  
List: nullptr nullptr  
1.insertAtBegin 2.insertAtEnd 3.insertAtPos 4.deleteAtBegin 5.deleteAtEnd  
6.deleteAtPos 7.search 8.reverse 0.Display -1.Exit  
Enter your choice: -1  
Exiting...  
  
notamitgamer@fedora:~/bsc/semester_2/assignments on 7 main !?  
~ > -
```

Discussion:

- **insAtBegin/insAtEnd/insAtPos** create a new node and link it into the chain by updating both the **next** pointer of the preceding node and the **prev** pointer of the following node, unlike a singly linked list which only needs one direction updated.
- **delAtBegin/delAtEnd/delAtPos** unlink a node by connecting its neighboring nodes directly to each other via their **prev** and **next** pointers before freeing it, handling edge cases for an empty or single-node list.
- **search()** traverses the list sequentially from **head**, comparing each node's data with the key, and reports the 1-based position if found.
- **reverse()** walks through the entire list once, swapping each node's **prev** and **next** pointers, then updates **head** to what was previously the last node — reversing the traversal direction in place without allocating new nodes.

Assignment 6

Problem Statement:

Implement a Circular Linked List. Include functions for insertion, deletion and search of a number, and reverse the list.

Problem Definition:

A **Circular Linked List** is a linear structure where the last node's `next` points back to the head instead of `NULL`, forming a loop. Insertion/deletion require tracking the last node to preserve the loop; traversal loops must use `do-while` -style checks (`temp != head` after moving) to avoid infinite loops.

Algorithm:

Insertion at Beginning:

```
procedure insertAtBegin(head, val)
begin
    newNode ← create new node;
    newNode.data ← val;
    if(head = NULL)
        head ← newNode;
        newNode.next ← head;
    else
        temp ← head;
        while(temp.next ≠ head) do
            temp ← temp.next;
        end while
        newNode.next ← head;
        temp.next ← newNode;
        head ← newNode;
    end if
end procedure
```

Insertion at End:

```
procedure insertAtEnd(head, val)
begin
    newNode ← create new node;
    newNode.data ← val;
    if(head = NULL)
        head ← newNode;
        newNode.next ← head;
    else
        temp ← head;
        while(temp.next ≠ head) do
            temp ← temp.next;
        end while
        temp.next ← newNode;
    end if
end procedure
```

```
        newNode.next ← head;
    end if
end procedure
```

Insertion at Position:

```
procedure insertAtPos(head, pos, val)
begin
    if(pos < 1)
        print "Invalid Position";
        return;
    end if
    if(pos = 1)
        call insertAtBegin(head, val);
        return;
    end if
    if(head = NULL)
        print "Out of range";
        return;
    end if
    temp ← head;
    for i ← 1 to pos - 2 do
        temp ← temp.next;
        if(temp = head)
            print "Out of range";
            return;
        end if
    end for
    newNode ← create new node;
    newNode.data ← val;
    newNode.next ← temp.next;
    temp.next ← newNode;
end procedure
```

Deletion at Beginning:

```
procedure deleteAtBegin(head)
begin
    if(head = NULL)
        print "List is empty";
        return;
    end if
    if(head.next = head)
        free head;
        head ← NULL;
        return;
    end if
    temp ← head;
```

```

        last ← head;
        while(last.next ≠ head) do
            last ← last.next;
        end while
        head ← head.next;
        last.next ← head;
        free temp;
    end procedure

```

Deletion at End:

```

procedure deleteAtEnd(head)
begin
    if(head = NULL)
        print "List is empty";
        return;
    end if
    if(head.next = head)
        free head;
        head ← NULL;
        return;
    end if
    temp ← head;
    while(temp.next.next ≠ head) do
        temp ← temp.next;
    end while
    delNode ← temp.next;
    temp.next ← head;
    free delNode;
end procedure

```

Deletion at Position:

```

procedure deleteAtPos(head, pos)
begin
    if(head = NULL or pos < 1)
        print "Unable to process";
        return;
    end if
    if(pos = 1)
        call deleteAtBegin(head);
        return;
    end if
    temp ← head;
    for i ← 1 to pos - 2 do
        temp ← temp.next;
        if(temp = head)
            print "Out of range";

```

```

        return;
    end if
end for
if(temp.next = head)
    print "Out of range";
    return;
end if
delNode ← temp.next;
temp.next ← delNode.next;
free delNode;
end procedure

```

Search:

```

procedure search(head, key)
begin
    if(head = NULL)
        return -1;
    end if
    temp ← head;
    pos ← 1;
    repeat
        if(temp.data = key)
            return pos;
        end if
        temp ← temp.next;
        pos ← pos + 1;
    until temp = head;
    return -1;
end procedure

```

Reverse:

```

procedure reverse(head)
begin
    if(head = NULL or head.next = head)
        return;
    end if
    prev ← NULL;
    curr ← head;
    nxt ← NULL;
    oldHead ← head;
    repeat
        nxt ← curr.next;
        curr.next ← prev;
        prev ← curr;
        curr ← nxt;
    until curr = head;

```

```
        oldHead.next ← prev;
        head ← prev;
end procedure
```

Source Code:

```
#include <iostream>
using namespace std;

struct node {
    int data;
    node *next = nullptr;
};

class CircularLinkedList {
private:
    node *head = nullptr;

public:
    void insertAtBegin(int val) {
        node *newNode = new node{val};
        if (head == nullptr) { head = newNode; newNode->next = head; return; }
        node *temp = head;
        while (temp->next != head) temp = temp->next;
        newNode->next = head;
        temp->next = newNode;
        head = newNode;
    }

    void insertAtEnd(int val) {
        node *newNode = new node{val};
        if (head == nullptr) { head = newNode; newNode->next = head; return; }
        node *temp = head;
        while (temp->next != head) temp = temp->next;
        temp->next = newNode;
        newNode->next = head;
    }

    void insertAtPos(int pos, int val) {
        if (pos < 1) { cout << "Invalid Position!\n"; return; }
        if (pos == 1) { insertAtBegin(val); return; }
        if (head == nullptr) { cout << "Out of range!\n"; return; }
        node *temp = head;
        for (int i = 1; i < pos - 1; i++) {
            temp = temp->next;
            if (temp == head) { cout << "Out of range!\n"; return; }
        }
        node *newNode = new node{val};
```

```

        newNode->next = temp->next;
        temp->next = newNode;
    }

    void deleteAtBegin() {
        if (head == nullptr) { cout << "List is empty!\n"; return; }
        if (head->next == head) { delete head; head = nullptr; return; }
        node *temp = head, *last = head;
        while (last->next != head) last = last->next;
        head = head->next;
        last->next = head;
        delete temp;
    }

    void deleteAtEnd() {
        if (head == nullptr) { cout << "List is empty!\n"; return; }
        if (head->next == head) { delete head; head = nullptr; return; }
        node *temp = head;
        while (temp->next->next != head) temp = temp->next;
        node *delNode = temp->next;
        temp->next = head;
        delete delNode;
    }

    void deleteAtPos(int pos) {
        if (head == nullptr || pos < 1) { cout << "Unable to process!\n"; return; }
        if (pos == 1) { deleteAtBegin(); return; }
        node *temp = head;
        for (int i = 1; i < pos - 1; i++) {
            temp = temp->next;
            if (temp == head) { cout << "Out of range!\n"; return; }
        }
        if (temp->next == head) { cout << "Out of range!\n"; return; }
        node *delNode = temp->next;
        temp->next = delNode->next;
        delete delNode;
    }

    int search(int key) {
        if (head == nullptr) return -1;
        node *temp = head;
        int pos = 1;
        do {
            if (temp->data == key) return pos;
            temp = temp->next;
            pos++;
        } while (temp != head);
        return -1;
    }

```

```

}

void reverse() {
    if (head == nullptr || head->next == head) return;
    node *prev = nullptr, *curr = head, *nxt = nullptr, *oldHead = head;
    do {
        nxt = curr->next;
        curr->next = prev;
        prev = curr;
        curr = nxt;
    } while (curr != head);
    oldHead->next = prev;
    head = prev;
}

void display() {
    if (head == nullptr) { cout << "List is empty!\n"; return; }
    node *temp = head;
    cout << "List: ";
    do { cout << temp->data << " -> "; temp = temp->next; } while (temp != head);
    cout << "(head)\n";
}

void printMenu() {
    cout << "1.insertAtBegin 2.insertAtEnd 3.insertAtPos 4.deleteAtBegin 5.deleteAtEnd\n"
    << "6.deleteAtPos 7.search 8.reverse 0.Display -1.Exit\n";
}

~CircularLinkedList() {
    if (head == nullptr) return;
    node *temp = head->next;
    while (temp != head) { node *next = temp->next; delete temp; temp = next; }
    delete head;
}

};

int main() {
    CircularLinkedList list;
    int choice, val, pos;

    while (true) {
        list.printMenu();
        cout << "Enter your choice: ";
        cin >> choice;

        switch (choice) {
            case 1: cout << "Enter value: "; cin >> val; list.insertAtBegin(val); break;

```

```

    case 2: cout << "Enter value: "; cin >> val; list.insertAtEnd(val); break;
    case 3:
        cout << "Enter position: "; cin >> pos;
        cout << "Enter value: "; cin >> val;
        list.insertAtPos(pos, val);
        break;
    case 4: list.deleteAtBegin(); break;
    case 5: list.deleteAtEnd(); break;
    case 6: cout << "Enter position: "; cin >> pos; list.deleteAtPos(pos); break;
    case 7: {
        cout << "Enter value to search: "; cin >> val;
        int res = list.search(val);
        if (res != -1) cout << "Found at position " << res << ".\n";
        else cout << "Not found.\n";
        break;
    }
    case 8: list.reverse(); break;
    case 0: list.display(); break;
    case -1: cout << "Exiting...\n"; return 0;
    default: cout << "Wrong choice. Try again.\n";
}
}
}

```

Output:

```
notamitgamer@fedora:~/bsc/semester_2/assignments on 7 main X!?  
~ > g++ assignment_05.cpp  
  
notamitgamer@fedora:~/bsc/semester_2/assignments on 7 main X!?  
~ > ./a.out  
1.insertAtBegin 2.insertAtEnd 3.insertAtPos 4.deleteAtBegin 5.deleteAtEnd  
6.deleteAtPos 7.search 8.reverse 0.Display -1.Exit  
Enter your choice: 1  
Enter value: 1  
1.insertAtBegin 2.insertAtEnd 3.insertAtPos 4.deleteAtBegin 5.deleteAtEnd  
6.deleteAtPos 7.search 8.reverse 0.Display -1.Exit  
Enter your choice: 2  
Enter value: 2  
1.insertAtBegin 2.insertAtEnd 3.insertAtPos 4.deleteAtBegin 5.deleteAtEnd  
6.deleteAtPos 7.search 8.reverse 0.Display -1.Exit  
Enter your choice: 3  
Enter position: 1  
Enter value: 0  
1.insertAtBegin 2.insertAtEnd 3.insertAtPos 4.deleteAtBegin 5.deleteAtEnd  
6.deleteAtPos 7.search 8.reverse 0.Display -1.Exit  
Enter your choice: 0  
List: 0 -> 1 -> 2 -> (head)  
1.insertAtBegin 2.insertAtEnd 3.insertAtPos 4.deleteAtBegin 5.deleteAtEnd  
6.deleteAtPos 7.search 8.reverse 0.Display -1.Exit  
Enter your choice: 7  
Enter value to search: 2  
Found at position 3.  
1.insertAtBegin 2.insertAtEnd 3.insertAtPos 4.deleteAtBegin 5.deleteAtEnd  
6.deleteAtPos 7.search 8.reverse 0.Display -1.Exit  
Enter your choice: 7  
Enter value to search: 5  
Not found.  
1.insertAtBegin 2.insertAtEnd 3.insertAtPos 4.deleteAtBegin 5.deleteAtEnd  
6.deleteAtPos 7.search 8.reverse 0.Display -1.Exit  
Enter your choice: 8  
1.insertAtBegin 2.insertAtEnd 3.insertAtPos 4.deleteAtBegin 5.deleteAtEnd  
6.deleteAtPos 7.search 8.reverse 0.Display -1.Exit  
Enter your choice: 0  
List: 2 -> 1 -> 0 -> (head)  
1.insertAtBegin 2.insertAtEnd 3.insertAtPos 4.deleteAtBegin 5.deleteAtEnd  
6.deleteAtPos 7.search 8.reverse 0.Display -1.Exit  
Enter your choice: 4  
1.insertAtBegin 2.insertAtEnd 3.insertAtPos 4.deleteAtBegin 5.deleteAtEnd  
6.deleteAtPos 7.search 8.reverse 0.Display -1.Exit  
Enter your choice: 5  
1.insertAtBegin 2.insertAtEnd 3.insertAtPos 4.deleteAtBegin 5.deleteAtEnd  
6.deleteAtPos 7.search 8.reverse 0.Display -1.Exit  
Enter your choice: 0  
List: 1 -> (head)  
1.insertAtBegin 2.insertAtEnd 3.insertAtPos 4.deleteAtBegin 5.deleteAtEnd  
6.deleteAtPos 7.search 8.reverse 0.Display -1.Exit  
Enter your choice: 6  
  
Enter position: 1  
1.insertAtBegin 2.insertAtEnd 3.insertAtPos 4.deleteAtBegin 5.deleteAtEnd  
6.deleteAtPos 7.search 8.reverse 0.Display -1.Exit  
Enter your choice: 0  
List is empty!  
1.insertAtBegin 2.insertAtEnd 3.insertAtPos 4.deleteAtBegin 5.deleteAtEnd  
6.deleteAtPos 7.search 8.reverse 0.Display -1.Exit  
Enter your choice: -1  
Exiting...  
  
notamitgamer@fedora:~/bsc/semester_2/assignments on 7 main X!?  
~ > -
```

Discussion:

- **insertAtBegin/insertAtEnd/insertAtPos** create a new node and link it in, always ensuring the last node's `next` still points to the (possibly updated) `head` to preserve the circle.
- **deleteAtBegin/deleteAtEnd/deleteAtPos** unlink a node by finding its neighbors and relinking around it, with the last node's `next` updated to the new head where relevant.
- **search()** traverses using a `do-while` loop (since `head` itself must be checked first) and stops once it loops back to `head`.
- **reverse()** performs a standard 3-pointer reversal (`prev` , `curr` , `nxt`) but loops until `curr` returns to the original `head` , then fixes the old head's `next` to close the circle with the new head.

Assignment 7

Problem Statement:

Perform Stack operations using Linked List implementation.

Problem Definition:

A **Stack** is a linear data structure that follows LIFO (Last In First Out) order. In a linked list implementation, each element is stored as a **node** containing data and a pointer to the next node, with the **top** pointer always referring to the most recently inserted node.

- **Push:** Insert a new node at the top.
- **Pop:** Remove the node from the top.
- **Peek:** View the top element without removing it.

Algorithm:

Check Empty:

```
procedure isEmpty(top)
begin
    if(top = NULL)
        return true;
    else
        return false;
    end if
end procedure
```

Push Operation:

```
procedure push(top, val)
begin
    newNode ← create new node;
    newNode.data ← val;
    newNode.next ← top;
    top ← newNode;
end procedure
```

Pop Operation:

```
procedure pop(top)
begin
    if(isEmpty(top))
        print "Stack Underflow";
        return;
    end if
    temp ← top;
    val ← temp.data;
```

```
        top ← top.next;
        free temp;
        return val;
    end procedure
```

Peek Operation:

```
procedure peek(top)
begin
    if(isEmpty(top))
        print "Stack is empty";
        return;
    end if
    return top.data;
end procedure
```

Source Code:

```
#include <iostream>
using namespace std;

struct node
{
    int data;
    node *next;
};

class Stack
{
private:
    node *top;

public:
    Stack() { top = nullptr; }

    bool isEmpty() { return (top == nullptr); }

    void push(int val)
    {
        node *newNode = new node;
        newNode->data = val;
        newNode->next = top;
        top = newNode;
    }

    int pop()
    {
        if (isEmpty())
```

```

        {
            cout << "Stack Underflow!\n";
            return -1;
        }
        node *temp = top;
        int val = temp->data;
        top = top->next;
        delete temp;
        return val;
    }

    int peek()
    {
        if (isEmpty())
        {
            cout << "Stack is empty!\n";
            return -1;
        }
        return top->data;
    }

    void display()
    {
        if (isEmpty())
        {
            cout << "Stack is empty!\n";
            return;
        }
        node *temp = top;
        while (temp != nullptr)
        {
            cout << temp->data << " ";
            temp = temp->next;
        }
        cout << endl;
    }

    ~Stack()
    {
        node *temp;
        while (top != nullptr)
        {
            temp = top;
            top = top->next;
            delete temp;
        }
    }
};

```

```

int main()
{
    Stack s;
    int choice, val;

    while (true)
    {
        cout << "1.push 2.pop 3.peek 4.isEmpty 0.Display -1.Exit\n";
        cout << "Enter your choice: ";
        cin >> choice;

        switch (choice)
        {
            case 1:
                cout << "Enter value: ";
                cin >> val;
                s.push(val);
                break;

            case 2:
            {
                int v = s.pop();
                if (v != -1)
                    cout << "Popped value: " << v << endl;
                break;
            }

            case 3:
            {
                int v = s.peek();
                if (v != -1)
                    cout << "Top element: " << v << endl;
                break;
            }

            case 4:
                cout << (s.isEmpty() ? "Empty\n" : "Not empty\n");
                break;

            case 0:
                s.display();
                break;

            case -1:
                cout << "Exiting...\n";
                return 0;
        }
    }
}

```

```

        default:
            cout << "Wrong choice.\n";
        }
    }
}

```

Output:

```

notamitgamer@fedora:~/bsc/semester_2/assignments on 7 main X!?
~ ) g++ assignment_06.cpp

notamitgamer@fedora:~/bsc/semester_2/assignments on 7 main X!?
~ ) ./a.out
1.push 2.pop 3.peek 4.isEmpty 0.Display -1.Exit
Enter your choice: 0
Stack is empty!
1.push 2.pop 3.peek 4.isEmpty 0.Display -1.Exit
Enter your choice: 1
Enter value: 1
1.push 2.pop 3.peek 4.isEmpty 0.Display -1.Exit
Enter your choice: 1
Enter value: 2
1.push 2.pop 3.peek 4.isEmpty 0.Display -1.Exit
Enter your choice: 1
Enter value: 3
1.push 2.pop 3.peek 4.isEmpty 0.Display -1.Exit
Enter your choice: 0
3 2 1
1.push 2.pop 3.peek 4.isEmpty 0.Display -1.Exit
Enter your choice: 2
Popped value: 3
1.push 2.pop 3.peek 4.isEmpty 0.Display -1.Exit
Enter your choice: 0
2 1
1.push 2.pop 3.peek 4.isEmpty 0.Display -1.Exit
Enter your choice: 3
Top element: 2
1.push 2.pop 3.peek 4.isEmpty 0.Display -1.Exit
Enter your choice: 4
Not empty
1.push 2.pop 3.peek 4.isEmpty 0.Display -1.Exit
Enter your choice: -1
Exiting...

notamitgamer@fedora:~/bsc/semester_2/assignments on 7 main X!?
~ ) -

```

Discussion:

- **push()** creates a new node, links it to the current top, and updates `top` to point to this new node.
- **pop()** checks for underflow, then removes the node pointed to by `top`, updates `top` to the next node, and returns the removed value.
- **peek()** returns the data stored at `top` without modifying the stack.

Assignment 8

Problem Statement:

Perform Stack operations using Array implementation.

Problem Definition:

A **Stack** is a linear data structure that follows LIFO (Last In First Out) order. Elements are added and removed only from one end, called the **top**.

- **Push:** Insert an element at the top.
- **Pop:** Remove the element from the top.
- **Peek:** View the top element without removing it.

Algorithm:

Check Full:

```
procedure isFull(top, capacity)
begin
    if(top = capacity - 1)
        return true;
    else
        return false;
    end if
end procedure
```

Check Empty:

```
procedure isEmpty(top)
begin
    if(top = -1)
        return true;
    else
        return false;
    end if
end procedure
```

Push Operation:

```
procedure push(arr, top, capacity, val)
begin
    if(isFull(top, capacity))
        print "Stack Overflow";
        return;
    end if
    top ← top + 1;
    arr[top] ← val;
```

```
end procedure
```

Pop Operation:

```
procedure pop(arr, top)
begin
    if(isEmpty(top))
        print "Stack Underflow";
        return;
    end if
    val ← arr[top];
    top ← top - 1;
    return val;
end procedure
```

Peek Operation:

```
procedure peek(arr, top)
begin
    if(isEmpty(top))
        print "Stack is empty";
        return;
    end if
    return arr[top];
end procedure
```

Source Code:

```
#include <iostream>
using namespace std;

class Stack {
private:
    int *arr;
    int top;
    int capacity;

public:
    Stack(int size) {
        capacity = size;
        arr = new int[capacity];
        top = -1;
    }

    bool isFull() { return (top == capacity - 1); }
    bool isEmpty() { return (top == -1); }

    void push() {
```

```

        if (isFull()) { cout << "Stack Overflow!\n"; return; }
        int val;
        cout << "Enter the value: ";
        cin >> val;
        arr[++top] = val;
    }

    void pop() {
        if (isEmpty()) { cout << "Stack Underflow!\n"; return; }
        int val = arr[top--];
        cout << "Popped value: " << val << endl;
    }

    void peek() {
        if (isEmpty()) { cout << "Stack is empty!\n"; return; }
        cout << "Top element: " << arr[top] << endl;
    }

    void display() {
        if (isEmpty()) { cout << "Stack is empty!\n"; return; }
        for (int i = top; i >= 0; i--) cout << arr[i] << " ";
        cout << endl;
    }

    ~Stack() { delete[] arr; }
};

int main() {
    int size, choice;
    cout << "Enter stack capacity: ";
    cin >> size;
    if (size <= 0) { cout << "Invalid size!\n"; return 0; }

    Stack s(size);
    while (true) {
        cout << "1.push 2.pop 3.peek 4.isFull 5.isEmpty 0.Display -1.Exit\n";
        cout << "Enter your choice: ";
        cin >> choice;

        switch (choice) {
            case 1: s.push(); break;
            case 2: s.pop(); break;
            case 3: s.peek(); break;
            case 4: cout << (s.isFull() ? "Full\n" : "Not full\n"); break;
            case 5: cout << (s.isEmpty() ? "Empty\n" : "Not empty\n"); break;
            case 0: s.display(); break;
            case -1: cout << "Exiting...\n"; return 0;
            default: cout << "Wrong choice.\n";
        }
    }
}

```

```

    }
}
}

```

Output:

```

notamitgamer@fedora:~/bsc/semester_2/assignments on 7 main X!?
~ ) g++ assignment_07.cpp

notamitgamer@fedora:~/bsc/semester_2/assignments on 7 main X!?
~ ) ./a.out
Enter stack capacity: 5
1.push 2.pop 3.peek 4.isFull 5.isEmpty 0.Display -1.Exit
Enter your choice: 1
Enter the value: 3
1.push 2.pop 3.peek 4.isFull 5.isEmpty 0.Display -1.Exit
Enter your choice: 1
Enter the value: 2
1.push 2.pop 3.peek 4.isFull 5.isEmpty 0.Display -1.Exit
Enter your choice: 1
Enter the value: 1
1.push 2.pop 3.peek 4.isFull 5.isEmpty 0.Display -1.Exit
Enter your choice: 0
1 2 3
1.push 2.pop 3.peek 4.isFull 5.isEmpty 0.Display -1.Exit
Enter your choice: 3
Top element: 1
1.push 2.pop 3.peek 4.isFull 5.isEmpty 0.Display -1.Exit
Enter your choice: 4
Not full
1.push 2.pop 3.peek 4.isFull 5.isEmpty 0.Display -1.Exit
Enter your choice: 5
Not empty
1.push 2.pop 3.peek 4.isFull 5.isEmpty 0.Display -1.Exit
Enter your choice: 2
Popped value: 1
1.push 2.pop 3.peek 4.isFull 5.isEmpty 0.Display -1.Exit
Enter your choice: 2
Popped value: 2
1.push 2.pop 3.peek 4.isFull 5.isEmpty 0.Display -1.Exit
Enter your choice: 0
3
1.push 2.pop 3.peek 4.isFull 5.isEmpty 0.Display -1.Exit
Enter your choice: -1
Exiting...

notamitgamer@fedora:~/bsc/semester_2/assignments on 7 main X!?
~ ) -

```

Discussion:

- **push()** checks for overflow, then increments `top` and stores the value at `arr[top]`.
- **pop()** checks for underflow, then returns `arr[top]` and decrements `top`, effectively removing it logically without clearing memory.
- **peek()** returns `arr[top]` without modifying `top`, allowing the caller to inspect the top element.

Assignment 9

Problem Statement:

Perform Queue operations using Array.

Problem Definition:

A **Queue** is a linear data structure that follows FIFO (First In First Out) order. Elements are inserted from the **rear** and removed from the **front**.

- **Enqueue:** Insert an element at the rear.
- **Dequeue:** Remove an element from the front.
- **Peek:** View the front element without removing it.

Algorithm:

Check Full:

```
procedure isFull(rear, capacity)
begin
    if(rear = capacity - 1)
        return true;
    else
        return false;
    end if
end procedure
```

Check Empty:

```
procedure isEmpty(front, rear)
begin
    if(front = -1 or front > rear)
        return true;
    else
        return false;
    end if
end procedure
```

Enqueue Operation:

```
procedure enqueue(arr, front, rear, capacity, val)
begin
    if(isFull(rear, capacity))
        print "Queue Overflow";
        return;
    end if
    if(front = -1)
        front ← 0;
```

```

        end if
        rear ← rear + 1;
        arr[rear] ← val;
    end procedure

```

Dequeue Operation:

```

procedure dequeue(arr, front, rear)
begin
    if(isEmpty(front, rear))
        print "Queue Underflow";
        return;
    end if
    val ← arr[front];
    front ← front + 1;
    if(front > rear)
        front ← -1;
        rear ← -1;
    end if
    return val;
end procedure

```

Peek Operation:

```

procedure peek(arr, front, rear)
begin
    if(isEmpty(front, rear))
        print "Queue is empty";
        return;
    end if
    return arr[front];
end procedure

```

Source Code:

```

#include <iostream>
using namespace std;

class Queue {
private:
    int *arr;
    int front, rear, capacity;

public:
    Queue(int size) {
        capacity = size;
        arr = new int[capacity];
        front = -1;
    }

```

```

        rear = -1;
    }

    bool isFull() { return (rear == capacity - 1); }
    bool isEmpty() { return (front == -1 || front > rear); }

    void enqueue() {
        if (isFull()) { cout << "Queue Overflow!\n"; return; }
        int val;
        cout << "Enter the value: ";
        cin >> val;
        if (front == -1) front = 0;
        arr[++rear] = val;
    }

    void dequeue() {
        if (isEmpty()) { cout << "Queue Underflow!\n"; return; }
        int val = arr[front++];
        if (front > rear) { front = -1; rear = -1; }
        cout << "Dequeued value: " << val << endl;
    }

    void peek() {
        if (isEmpty()) { cout << "Queue is empty!\n"; return; }
        cout << "Front element: " << arr[front] << endl;
    }

    void display() {
        if (isEmpty()) { cout << "Queue is empty!\n"; return; }
        for (int i = front; i <= rear; i++) cout << arr[i] << " ";
        cout << endl;
    }

    ~Queue() { delete[] arr; }
};

int main() {
    int size, choice;
    cout << "Enter queue capacity: ";
    cin >> size;
    if (size <= 0) { cout << "Invalid size!\n"; return 0; }

    Queue q(size);
    while (true) {
        cout << "1.enqueue 2.dequeue 3.peek 4.isFull 5.isEmpty 0.Display -1.Exit\n";
        cout << "Enter your choice: ";
        cin >> choice;
    }
}

```

```

switch (choice) {
    case 1: q.enqueue(); break;
    case 2: q.dequeue(); break;
    case 3: q.peek(); break;
    case 4: cout << (q.isFull() ? "Full\n" : "Not full\n"); break;
    case 5: cout << (q.isEmpty() ? "Empty\n" : "Not empty\n"); break;
    case 0: q.display(); break;
    case -1: cout << "Exiting...\n"; return 0;
    default: cout << "Wrong choice.\n";
}
}
}

```

Output:

```

notamitgamer@fedora:~/bsc/semester_2/assignments on ʘ main X!ʘ
~ ) g++ assignment_08.cpp

notamitgamer@fedora:~/bsc/semester_2/assignments on ʘ main X!ʘ
~ ) ./a.out
Enter queue capacity: 5
1.enqueue 2.dequeue 3.peek 4.isFull 5.isEmpty 0.Display -1.Exit
Enter your choice: 1
Enter the value: 3
1.enqueue 2.dequeue 3.peek 4.isFull 5.isEmpty 0.Display -1.Exit
Enter your choice: 1
Enter the value: 2
1.enqueue 2.dequeue 3.peek 4.isFull 5.isEmpty 0.Display -1.Exit
Enter your choice: 1
Enter the value: 1
1.enqueue 2.dequeue 3.peek 4.isFull 5.isEmpty 0.Display -1.Exit
Enter your choice: 0
3 2 1
1.enqueue 2.dequeue 3.peek 4.isFull 5.isEmpty 0.Display -1.Exit
Enter your choice: 4
Not full
1.enqueue 2.dequeue 3.peek 4.isFull 5.isEmpty 0.Display -1.Exit
Enter your choice: 5
Not empty
1.enqueue 2.dequeue 3.peek 4.isFull 5.isEmpty 0.Display -1.Exit
Enter your choice: 3
Front element: 3
1.enqueue 2.dequeue 3.peek 4.isFull 5.isEmpty 0.Display -1.Exit
Enter your choice: 2
Dequeued value: 3
1.enqueue 2.dequeue 3.peek 4.isFull 5.isEmpty 0.Display -1.Exit
Enter your choice: 2
Dequeued value: 2
1.enqueue 2.dequeue 3.peek 4.isFull 5.isEmpty 0.Display -1.Exit
Enter your choice: 2
Dequeued value: 1
1.enqueue 2.dequeue 3.peek 4.isFull 5.isEmpty 0.Display -1.Exit
Enter your choice: 0
Queue is empty!
1.enqueue 2.dequeue 3.peek 4.isFull 5.isEmpty 0.Display -1.Exit
Enter your choice: -1
Exiting...

notamitgamer@fedora:~/bsc/semester_2/assignments on ʘ main X!ʘ
~ )

```

Discussion:

- **enqueue()** checks for overflow, initializes `front` to 0 on the first insertion, then places the value at `arr[++rear]`.
- **dequeue()** checks for underflow, returns `arr[front]`, increments `front`, and resets both `front` and `rear` to `-1` once the queue empties — since this linear queue does not reuse freed front slots.
- **peek()** returns `arr[front]` without modifying either pointer.

Assignment 10

Problem Statement:

Perform Circular Queue operations using Array.

Problem Definition:

A **Circular Queue** is a queue implemented using a fixed-size array where the rear wraps around to the front once it reaches the end, using modulo arithmetic. This allows reuse of freed slots, unlike a linear queue.

Algorithm:

Check Empty:

```
procedure isEmpty(front)
begin
    if(front = -1)
        return true;
    else
        return false;
    end if
end procedure
```

Check Full:

```
procedure isFull(front, rear, size)
begin
    if((rear + 1) mod size = front)
        return true;
    else
        return false;
    end if
end procedure
```

Enqueue Operation:

```
procedure enqueue(queue, front, rear, size, val)
begin
    if(isFull(front, rear, size))
        print "Queue is Full";
        return;
    end if
    if(isEmpty(front))
        front ← 0;
        rear ← 0;
    else
        rear ← (rear + 1) mod size;
    end if
```

```
        queue[rear] ← val;
end procedure
```

Dequeue Operation:

```
procedure dequeue(queue, front, rear, size)
begin
    if(isEmpty(front))
        print "Queue is Empty";
        return;
    end if
    val ← queue[front];
    if(front = rear)
        front ← -1;
        rear ← -1;
    else
        front ← (front + 1) mod size;
    end if
    return val;
end procedure
```

Peek Operation:

```
procedure peek(queue, front)
begin
    if(isEmpty(front))
        print "Queue is Empty";
        return;
    end if
    return queue[front];
end procedure
```

Source Code:

```
#include <iostream>
using namespace std;

class CircularQueue {
private:
    int *queue;
    int front, rear, size;

public:
    CircularQueue(int n) {
        size = n;
        queue = new int[size];
        front = -1;
        rear = -1;
    }
};
```

```

}

bool isEmpty() { return front == -1; }
bool isFull() { return (rear + 1) % size == front; }

void enqueue(int val) {
    if (isFull()) { cout << "Queue is Full!\n"; return; }
    if (isEmpty()) { front = 0; rear = 0; }
    else rear = (rear + 1) % size;
    queue[rear] = val;
}

void dequeue() {
    if (isEmpty()) { cout << "Queue is Empty!\n"; return; }
    int val = queue[front];
    if (front == rear) { front = -1; rear = -1; }
    else front = (front + 1) % size;
    cout << "Removed: " << val << endl;
}

void peek() {
    if (isEmpty()) { cout << "Queue is Empty!\n"; return; }
    cout << "Front element: " << queue[front] << endl;
}

void display() {
    if (isEmpty()) { cout << "Queue is Empty!\n"; return; }
    int i = front;
    while (true) {
        cout << queue[i] << " ";
        if (i == rear) break;
        i = (i + 1) % size;
    }
    cout << endl;
}

~CircularQueue() { delete[] queue; }
};

int main() {
    int n, choice, val;
    cout << "Enter queue size: ";
    cin >> n;
    if (n <= 0) { cout << "Invalid size!\n"; return 0; }

    CircularQueue q(n);
    while (true) {
        cout << "1.enqueue 2.dequeue 3.peek 4.isEmpty 5.isFull 6.Display 0.Exit\n";

```

```

cout << "Enter your choice: ";
cin >> choice;

switch (choice) {
    case 1: cout << "Enter value: "; cin >> val; q.enqueue(val); break;
    case 2: q.dequeue(); break;
    case 3: q.peek(); break;
    case 4: cout << (q.isEmpty() ? "Empty\n" : "Not empty\n"); break;
    case 5: cout << (q.isFull() ? "Full\n" : "Not full\n"); break;
    case 6: q.display(); break;
    case 0: cout << "Exiting...\n"; return 0;
    default: cout << "Wrong choice.\n";
}
}
}

```

Output:

```

notamitgamer@fedora:~/bsc/semester_2/assignments on 7 main X!?
~ > g++ assignment_09.cpp

notamitgamer@fedora:~/bsc/semester_2/assignments on 7 main X!?
~ > ./a.out
Enter queue size: 3
1.enqueue 2.dequeue 3.peek 4.isEmpty 5.isFull 6.Display 0.Exit
Enter your choice: 1
Enter value: 1
1.enqueue 2.dequeue 3.peek 4.isEmpty 5.isFull 6.Display 0.Exit
Enter your choice: 1
Enter value: 2
1.enqueue 2.dequeue 3.peek 4.isEmpty 5.isFull 6.Display 0.Exit
Enter your choice: 1
Enter value: 3
1.enqueue 2.dequeue 3.peek 4.isEmpty 5.isFull 6.Display 0.Exit
Enter your choice: 6
1 2 3
1.enqueue 2.dequeue 3.peek 4.isEmpty 5.isFull 6.Display 0.Exit
Enter your choice: 3
Front element: 1
1.enqueue 2.dequeue 3.peek 4.isEmpty 5.isFull 6.Display 0.Exit
Enter your choice: 4
Not empty
1.enqueue 2.dequeue 3.peek 4.isEmpty 5.isFull 6.Display 0.Exit
Enter your choice: 5
Full
1.enqueue 2.dequeue 3.peek 4.isEmpty 5.isFull 6.Display 0.Exit
Enter your choice: 2
Removed: 1
1.enqueue 2.dequeue 3.peek 4.isEmpty 5.isFull 6.Display 0.Exit
Enter your choice: 2
Removed: 2
1.enqueue 2.dequeue 3.peek 4.isEmpty 5.isFull 6.Display 0.Exit
Enter your choice: 6
3
1.enqueue 2.dequeue 3.peek 4.isEmpty 5.isFull 6.Display 0.Exit
Enter your choice: 0
Exiting...

notamitgamer@fedora:~/bsc/semester_2/assignments on 7 main X!?
~ > _

```

Discussion:

- **enqueue()** checks fullness using `(rear + 1) % size == front`, sets `front / rear` to `0` on the first insertion, otherwise advances `rear` circularly with modulo before storing the value.
- **dequeue()** returns `queue[front]`, and either resets both pointers to `-1` (if it was the last element) or advances `front` circularly, allowing wrapped-around slots to be reused.
- **peek()** simply returns `queue[front]` without altering any pointer.

Assignment 11

Problem Statement:

Perform Queue operations using Linked List.

Problem Definition:

A **Queue** implemented via a linked list uses two pointers, **front** and **rear**, to track the first and last nodes, allowing enqueue and dequeue in $O(1)$ without a fixed capacity.

Algorithm:

Check Empty:

```
procedure isEmpty(front)
begin
    if(front = NULL)
        return true;
    else
        return false;
    end if
end procedure
```

Enqueue Operation:

```
procedure enqueue(front, rear, val)
begin
    newNode ← create new node;
    newNode.data ← val;
    newNode.next ← NULL;
    if(isEmpty(front))
        front ← newNode;
        rear ← newNode;
    else
        rear.next ← newNode;
        rear ← newNode;
    end if
end procedure
```

Dequeue Operation:

```
procedure dequeue(front, rear)
begin
    if(isEmpty(front))
        print "Queue Underflow";
        return;
    end if
    temp ← front;
```

```

        val ← temp.data;
        front ← front.next;
        if(front = NULL)
            rear ← NULL;
        end if
        free temp;
        return val;
    end procedure
end procedure

```

Peek Operation:

```

procedure peek(front)
begin
    if(isEmpty(front))
        print "Queue is empty";
        return;
    end if
    return front.data;
end procedure

```

Source Code:

```

#include <iostream>
using namespace std;

struct node {
    int data;
    node *next = nullptr;
};

class Queue {
private:
    node *front, *rear;

public:
    Queue() { front = nullptr; rear = nullptr; }
    bool isEmpty() { return (front == nullptr); }

    void enqueue() {
        node *newNode = new node;
        cout << "Enter the value: ";
        cin >> newNode->data;
        newNode->next = nullptr;
        if (isEmpty()) front = rear = newNode;
        else { rear->next = newNode; rear = newNode; }
    }

    void dequeue() {

```

```

        if (isEmpty()) { cout << "Queue Underflow!\n"; return; }
        node *temp = front;
        int val = temp->data;
        front = front->next;
        if (front == nullptr) rear = nullptr;
        delete temp;
        cout << "Dequeued: " << val << endl;
    }

    void peek() {
        if (isEmpty()) { cout << "Queue is empty!\n"; return; }
        cout << "Front element: " << front->data << endl;
    }

    void display() {
        if (isEmpty()) { cout << "Queue is empty!\n"; return; }
        node *temp = front;
        while (temp != nullptr) { cout << temp->data << " "; temp = temp->next; }
        cout << endl;
    }

    ~Queue() {
        node *temp;
        while (front != nullptr) { temp = front; front = front->next; delete temp; }
    }
};

int main() {
    Queue q;
    int choice;

    while (true) {
        cout << "1.enqueue 2.dequeue 3.peek 4.isEmpty 0.Display -1.Exit\n";
        cout << "Enter your choice: ";
        cin >> choice;

        switch (choice) {
            case 1: q.enqueue(); break;
            case 2: q.dequeue(); break;
            case 3: q.peek(); break;
            case 4: cout << (q.isEmpty() ? "Empty\n" : "Not empty\n"); break;
            case 0: q.display(); break;
            case -1: cout << "Exiting...\n"; return 0;
            default: cout << "Wrong choice.\n";
        }
    }
}

```

Output:

```
notamitgamer@fedora:~/bsc/semester_2/assignments on  main X!?  
~  g++ assignment_10.cpp  
  
notamitgamer@fedora:~/bsc/semester_2/assignments on  main X!?  
~  ./a.out  
1.enqueue 2.dequeue 3.peek 4.isEmpty 0.Display -1.Exit  
Enter your choice: 1  
Enter the value: 1  
1.enqueue 2.dequeue 3.peek 4.isEmpty 0.Display -1.Exit  
Enter your choice: 1  
Enter the value: 2  
1.enqueue 2.dequeue 3.peek 4.isEmpty 0.Display -1.Exit  
Enter your choice: 0  
1 2  
1.enqueue 2.dequeue 3.peek 4.isEmpty 0.Display -1.Exit  
Enter your choice: 3  
Front element: 1  
1.enqueue 2.dequeue 3.peek 4.isEmpty 0.Display -1.Exit  
Enter your choice: 2  
Dequeued: 1  
1.enqueue 2.dequeue 3.peek 4.isEmpty 0.Display -1.Exit  
Enter your choice: 2  
Dequeued: 2  
1.enqueue 2.dequeue 3.peek 4.isEmpty 0.Display -1.Exit  
Enter your choice: 3  
Queue is empty!  
1.enqueue 2.dequeue 3.peek 4.isEmpty 0.Display -1.Exit  
Enter your choice: 4  
Empty  
1.enqueue 2.dequeue 3.peek 4.isEmpty 0.Display -1.Exit  
Enter your choice: 0  
Queue is empty!  
1.enqueue 2.dequeue 3.peek 4.isEmpty 0.Display -1.Exit  
Enter your choice: -1  
Exiting...  
  
notamitgamer@fedora:~/bsc/semester_2/assignments on  main X!?  
~  _
```

Discussion:

- **enqueue()** creates a new node and links it after `rear`, updating `rear` to point to it; if the queue was empty, both `front` and `rear` are set to the new node.
- **dequeue()** removes the node at `front`, advancing `front` to the next node, and resets `rear` to `NULL` if the list becomes empty.
- **peek()** returns `front->data` without modifying the queue.

Assignment 12

Problem Statement:

Create and perform different operations on Double-ended Queues (Deque) using Linked List implementation.

Problem Definition:

A **Deque (Double-Ended Queue)** allows insertion and deletion from both ends — front and rear. It is implemented here using a doubly linked list, so both `prev` and `next` pointers are updated during insert/delete at either end.

Algorithm:

Check Empty:

```
procedure isEmpty(front)
begin
    if(front = NULL)
        return true;
    else
        return false;
    end if
end procedure
```

Insert at Front:

```
procedure insertFront(front, rear, val)
begin
    newNode ← create new node;
    newNode.data ← val;
    newNode.prev ← NULL;
    newNode.next ← front;
    if(isEmpty(front))
        front ← newNode;
        rear ← newNode;
    else
        front.prev ← newNode;
        front ← newNode;
    end if
end procedure
```

Insert at Rear:

```
procedure insertRear(front, rear, val)
begin
    newNode ← create new node;
    newNode.data ← val;
    newNode.next ← NULL;
    newNode.prev ← rear;
```

```

        if(isEmpty(front))
            front ← newNode;
            rear ← newNode;
        else
            rear.next ← newNode;
            rear ← newNode;
        end if
    end procedure
end procedure

```

Delete from Front:

```

procedure deleteFront(front, rear)
begin
    if(isEmpty(front))
        print "Deque Underflow";
        return;
    end if
    temp ← front;
    val ← temp.data;
    front ← front.next;
    if(front = NULL)
        rear ← NULL;
    else
        front.prev ← NULL;
    end if
    free temp;
    return val;
end procedure

```

Delete from Rear:

```

procedure deleteRear(front, rear)
begin
    if(isEmpty(front))
        print "Deque Underflow";
        return;
    end if
    temp ← rear;
    val ← temp.data;
    rear ← rear.prev;
    if(rear = NULL)
        front ← NULL;
    else
        rear.next ← NULL;
    end if
    free temp;
    return val;
end procedure

```

Get Front / Get Rear:

```
procedure getFront(front)
begin
    if(isEmpty(front))
        print "Deque is empty";
        return;
    end if
    return front.data;
end procedure

procedure getRear(rear)
begin
    if(rear = NULL)
        print "Deque is empty";
        return;
    end if
    return rear.data;
end procedure
```

Source Code:

```
#include <iostream>
using namespace std;

struct node {
    int data;
    node *prev = nullptr;
    node *next = nullptr;
};

class Deque {
private:
    node *front, *rear;

public:
    Deque() { front = nullptr; rear = nullptr; }
    bool isEmpty() { return (front == nullptr); }

    void insertFront() {
        node *newNode = new node;
        cout << "Enter the value: ";
        cin >> newNode->data;
        newNode->prev = nullptr;
        newNode->next = front;
        if (isEmpty()) front = rear = newNode;
        else { front->prev = newNode; front = newNode; }
    }
}
```

```

void insertRear() {
    node *newNode = new node;
    cout << "Enter the value: ";
    cin >> newNode->data;
    newNode->next = nullptr;
    newNode->prev = rear;
    if (isEmpty()) front = rear = newNode;
    else { rear->next = newNode; rear = newNode; }
}

void deleteFront() {
    if (isEmpty()) { cout << "Deque Underflow!\n"; return; }
    node *temp = front;
    int val = temp->data;
    front = front->next;
    if (front == nullptr) rear = nullptr;
    else front->prev = nullptr;
    delete temp;
    cout << "Deleted from front: " << val << endl;
}

void deleteRear() {
    if (isEmpty()) { cout << "Deque Underflow!\n"; return; }
    node *temp = rear;
    int val = temp->data;
    rear = rear->prev;
    if (rear == nullptr) front = nullptr;
    else rear->next = nullptr;
    delete temp;
    cout << "Deleted from rear: " << val << endl;
}

void getFront() {
    if (isEmpty()) { cout << "Deque is empty!\n"; return; }
    cout << "Front element: " << front->data << endl;
}

void getRear() {
    if (isEmpty()) { cout << "Deque is empty!\n"; return; }
    cout << "Rear element: " << rear->data << endl;
}

void display() {
    if (isEmpty()) { cout << "Deque is empty!\n"; return; }
    node *temp = front;
    while (temp != nullptr) { cout << temp->data << " "; temp = temp->next; }
    cout << endl;
}

```

```

    }

    ~Deque() {
        node *temp;
        while (front != nullptr) { temp = front; front = front->next; delete temp; }
    }
};

int main() {
    Deque dq;
    int choice;

    while (true) {
        cout << "1.insertFront 2.insertRear 3.deleteFront 4.deleteRear\n"
              << "5.getFront 6.getRear 7.isEmpty 0.Display -1.Exit\n";
        cout << "Enter your choice: ";
        cin >> choice;

        switch (choice) {
            case 1: dq.insertFront(); break;
            case 2: dq.insertRear(); break;
            case 3: dq.deleteFront(); break;
            case 4: dq.deleteRear(); break;
            case 5: dq.getFront(); break;
            case 6: dq.getRear(); break;
            case 7: cout << (dq.isEmpty() ? "Empty\n" : "Not empty\n"); break;
            case 0: dq.display(); break;
            case -1: cout << "Exiting...\n"; return 0;
            default: cout << "Wrong choice.\n";
        }
    }
}

```

Output:

```
notamitgamer@fedora:~/bsc/semester_2/assignments on 7 main X!?  
~ ) g++ assignment_11.cpp  
  
notamitgamer@fedora:~/bsc/semester_2/assignments on 7 main X!?  
~ ) ./a.out  
1.insertFront 2.insertRear 3.deleteFront 4.deleteRear  
5.getFront 6.getRear 7.isEmpty 0.Display -1.Exit  
Enter your choice: 7  
Empty  
1.insertFront 2.insertRear 3.deleteFront 4.deleteRear  
5.getFront 6.getRear 7.isEmpty 0.Display -1.Exit  
Enter your choice: 1  
Enter the value: 1  
1.insertFront 2.insertRear 3.deleteFront 4.deleteRear  
5.getFront 6.getRear 7.isEmpty 0.Display -1.Exit  
Enter your choice: 2  
Enter the value: 2  
1.insertFront 2.insertRear 3.deleteFront 4.deleteRear  
5.getFront 6.getRear 7.isEmpty 0.Display -1.Exit  
Enter your choice: 0  
1 2  
1.insertFront 2.insertRear 3.deleteFront 4.deleteRear  
5.getFront 6.getRear 7.isEmpty 0.Display -1.Exit  
Enter your choice: 5  
Front element: 1  
1.insertFront 2.insertRear 3.deleteFront 4.deleteRear  
5.getFront 6.getRear 7.isEmpty 0.Display -1.Exit  
Enter your choice: 6  
Rear element: 2  
1.insertFront 2.insertRear 3.deleteFront 4.deleteRear  
5.getFront 6.getRear 7.isEmpty 0.Display -1.Exit  
Enter your choice: 3  
Deleted from front: 1  
1.insertFront 2.insertRear 3.deleteFront 4.deleteRear  
5.getFront 6.getRear 7.isEmpty 0.Display -1.Exit  
Enter your choice: 4  
Deleted from rear: 2  
1.insertFront 2.insertRear 3.deleteFront 4.deleteRear  
5.getFront 6.getRear 7.isEmpty 0.Display -1.Exit  
Enter your choice: 0  
Deque is empty!  
1.insertFront 2.insertRear 3.deleteFront 4.deleteRear  
5.getFront 6.getRear 7.isEmpty 0.Display -1.Exit  
Enter your choice: -1  
Exiting...  
  
notamitgamer@fedora:~/bsc/semester_2/assignments on 7 main X!?  
~ ) -
```

Discussion:

- **insertFront()/insertRear()** create a new node and link it at the respective end, updating both `prev` and `next` pointers of neighboring nodes to maintain the doubly linked structure.
- **deleteFront()/deleteRear()** remove the node at the respective end by advancing `front` or `rear` to the adjacent node and clearing the freed node's neighbor pointer, with `front / rear` both reset to `NULL` if the deque becomes empty.
- **getFront()/getRear()** simply return `front->data` or `rear->data` without modifying the deque.

Assignment 13

Problem Statement:

Write a program to scan a polynomial using a linked list and add two polynomials.

Problem Definition:

A **Polynomial** can be represented as a linked list of terms, where each node stores a **coeff** (coefficient) and **exp** (exponent), sorted in decreasing order of exponent.

- **insertTerm:** Inserts a term into the list in the correct sorted position, merging with an existing term of the same exponent.
- **Addition:** Two polynomials are added by traversing both lists simultaneously, comparing exponents, and combining coefficients of matching terms.

Algorithm:

Insert Term in Sorted Order:

```
procedure insertTerm(head, coeff, exp)
begin
    if(coeff = 0)
        return;
    end if
    newNode ← create new node;
    newNode.coeff ← coeff;
    newNode.exp ← exp;
    newNode.next ← NULL;
    if(head = NULL or exp > head.exp)
        newNode.next ← head;
        head ← newNode;
        return;
    end if
    if(head.exp = exp)
        head.coeff ← head.coeff + coeff;
        free newNode;
        if(head.coeff = 0)
            temp ← head;
            head ← head.next;
            free temp;
        end if
        return;
    end if
    temp ← head;
    while(temp.next ≠ NULL and temp.next.exp > exp) do
        temp ← temp.next;
    end while
    if(temp.next ≠ NULL and temp.next.exp = exp)
```

```

        temp.next.coeff ← temp.next.coeff + coeff;
        free newNode;
        if(temp.next.coeff = 0)
            delNode ← temp.next;
            temp.next ← delNode.next;
            free delNode;
        end if
    else
        newNode.next ← temp.next;
        temp.next ← newNode;
    end if
end procedure

```

Scan Polynomial:

```

procedure scanPolynomial(head)
begin
    read n;
    for i ← 1 to n do
        read coeff, exp;
        call insertTerm(head, coeff, exp);
    end for
end procedure

```

Add Polynomials:

```

procedure addPolynomials(head1, head2, resultHead)
begin
    p ← head1;
    q ← head2;
    while(p ≠ NULL and q ≠ NULL) do
        if(p.exp = q.exp)
            sum ← p.coeff + q.coeff;
            if(sum ≠ 0)
                call insertTerm(resultHead, sum, p.exp);
            end if
            p ← p.next;
            q ← q.next;
        else if(p.exp > q.exp)
            call insertTerm(resultHead, p.coeff, p.exp);
            p ← p.next;
        else
            call insertTerm(resultHead, q.coeff, q.exp);
            q ← q.next;
        end if
    end while
    while(p ≠ NULL) do
        call insertTerm(resultHead, p.coeff, p.exp);
    end while

```

```

        p ← p.next;
    end while
    while(q ≠ NULL) do
        call insertTerm(resultHead, q.coeff, q.exp);
        q ← q.next;
    end while
end procedure

```

Source Code:

```

#include <iostream>
#include <cmath>
using namespace std;

struct node {
    int coeff, exp;
    node *next = nullptr;
};

class Polynomial {
private:
    node *head;

    void clear() {
        node *temp;
        while (head != nullptr) {
            temp = head;
            head = head->next;
            delete temp;
        }
    }

    void copyFrom(const Polynomial &other) {
        if (other.head == nullptr) {
            head = nullptr;
            return;
        }
        head = new node{other.head->coeff, other.head->exp, nullptr};
        node *current = head;
        node *otherCurrent = other.head->next;
        while (otherCurrent != nullptr) {
            current->next = new node{otherCurrent->coeff, otherCurrent->exp, nullptr};
            current = current->next;
            otherCurrent = otherCurrent->next;
        }
    }

public:

```

```

Polynomial() { head = nullptr; }

Polynomial(const Polynomial &other) {
    copyFrom(other);
}

Polynomial& operator=(const Polynomial &other) {
    if (this != &other) {
        clear();
        copyFrom(other);
    }
    return *this;
}

~Polynomial() {
    clear();
}

void insertTerm(int coeff, int exp) {
    if (coeff == 0) return;
    node *newNode = new node{coeff, exp, nullptr};

    if (head == nullptr || exp > head->exp) {
        newNode->next = head;
        head = newNode;
        return;
    }
    if (head->exp == exp) {
        head->coeff += coeff;
        delete newNode;
        if (head->coeff == 0) {
            node* temp = head;
            head = head->next;
            delete temp;
        }
        return;
    }
    node *temp = head;
    while (temp->next != nullptr && temp->next->exp > exp) temp = temp->next;
    if (temp->next != nullptr && temp->next->exp == exp) {
        temp->next->coeff += coeff;
        delete newNode;
        if (temp->next->coeff == 0) {
            node* toDelete = temp->next;
            temp->next = temp->next->next;
            delete toDelete;
        }
    } else {

```

```

        newNode->next = temp->next;
        temp->next = newNode;
    }
}

void inputPolynomial() {
    int n, coeff, exp;
    cout << "Enter number of terms: ";
    cin >> n;
    for (int i = 0; i < n; i++) {
        cout << "Coefficient: "; cin >> coeff;
        cout << "Exponent: "; cin >> exp;
        insertTerm(coeff, exp);
    }
}

Polynomial add(const Polynomial &other) const {
    Polynomial result;
    node *p = head, *q = other.head;
    while (p != nullptr && q != nullptr) {
        if (p->exp == q->exp) {
            int sum = p->coeff + q->coeff;
            if (sum != 0) result.insertTerm(sum, p->exp);
            p = p->next; q = q->next;
        } else if (p->exp > q->exp) {
            result.insertTerm(p->coeff, p->exp);
            p = p->next;
        } else {
            result.insertTerm(q->coeff, q->exp);
            q = q->next;
        }
    }
    while (p != nullptr) { result.insertTerm(p->coeff, p->exp); p = p->next; }
    while (q != nullptr) { result.insertTerm(q->coeff, q->exp); q = q->next; }
    return result;
}

void display() const {
    if (head == nullptr) { cout << "0\n"; return; }
    node *temp = head;
    bool first = true;
    while (temp != nullptr) {
        if (!first && temp->coeff > 0) cout << " + ";
        else if (!first && temp->coeff < 0) cout << " - ";
        else if (temp->coeff < 0) cout << "-";
        int c = abs(temp->coeff);
        if (temp->exp == 0) cout << c;
        else if (temp->exp == 1) cout << (c == 1 ? "x" : to_string(c) + "x");
    }
}

```

```

        else cout << (c == 1 ? "x^" + to_string(temp->exp) : to_string(c) + "x^" +
to_string(temp->exp));
        first = false;
        temp = temp->next;
    }
    cout << endl;
}
};

int main() {
    Polynomial p1, p2, result;
    cout << "--- First Polynomial ---\n";
    p1.inputPolynomial();
    cout << "--- Second Polynomial ---\n";
    p2.inputPolynomial();

    cout << "First: "; p1.display();
    cout << "Second: "; p2.display();

    result = p1.add(p2);
    cout << "Sum: "; result.display();
    return 0;
}

```

Output:

```

notamitgamer@fedora:~/bsc/semester_2/assignments on 17 main X!?
~ > g++ assignment_12.cpp

notamitgamer@fedora:~/bsc/semester_2/assignments on 17 main X!?
~ > ./a.out
--- First Polynomial ---
Enter number of terms: 3
Coefficient: -2
Exponent: 2
Coefficient: 5
Exponent: 1
Coefficient: -76
Exponent: 0
--- Second Polynomial ---
Enter number of terms: 2
Coefficient: 25
Exponent: 1
Coefficient: 1
Exponent: 2
First: -2x^2 + 5x - 76
Second: x^2 + 25x
Sum: -x^2 + 30x - 76

notamitgamer@fedora:~/bsc/semester_2/assignments on 17 main X!?
~ > _

```

Discussion:

- **insertTerm()** inserts a term into the correct sorted-by-exponent position, merging its coefficient into an existing node if a term with the same exponent already exists, and discarding it entirely if the resulting coefficient is zero.
- **inputPolynomial()** reads the number of terms, then takes each coefficient and exponent from the user and passes them to `insertTerm()`, so the list is always kept sorted by exponent.
- **add()** walks both polynomial lists in parallel, comparing exponents at each step: matching exponents have their coefficients summed, while the term with the higher exponent is copied over as-is, until one list is exhausted and the rest of the other is appended — producing a new sorted result polynomial.

Assignment 14

Problem Statement:

Write a program to calculate factorial and to compute the factors of a given number using (i) recursion and (ii) iteration.

Problem Definition:

- **Factorial (n!):** Product of all positive integers up to n . Recursive definition: $n! = n * (n-1)!$, with base case $0! = 1! = 1$.
- **Factors:** Numbers that divide n exactly (remainder 0). Can be found either by recursively checking divisors from 1 to n , or with a simple iterative loop.

Algorithm:

Factorial using Recursion:

```
procedure factorialRecursive(n)
begin
    if(n <= 1)
        return 1;
    else
        return n * factorialRecursive(n - 1);
    end if
end procedure
```

Factorial using Iteration:

```
procedure factorialIterative(n)
begin
    fact ← 1;
    for i ← 2 to n do
        fact ← fact * i;
    end for
    return fact;
end procedure
```

Factors using Recursion:

```
procedure factorsRecursive(n, divisor)
begin
    if(divisor > n)
        return;
    end if
    if(n mod divisor = 0)
        print divisor + " ";
    end if
    call factorsRecursive(n, divisor + 1);
end procedure
```

```
end procedure
```

Factors using Iteration:

```
procedure factorsIterative(n)
begin
    for i ← 1 to n do
        if(n mod i = 0)
            print i + " ";
        end if
    end for
end procedure
```

Source Code:

```
#include <iostream>
using namespace std;

class MathOperations {
private:
    int num;

    long long factorialRecursive(int n) {
        if (n <= 1) return 1;
        return n * factorialRecursive(n - 1);
    }

    void factorsRecursive(int n, int divisor) {
        if (divisor > n) return;
        if (n % divisor == 0) cout << divisor << " ";
        factorsRecursive(n, divisor + 1);
    }

public:
    MathOperations(int n) { num = n; }

    void calculateFactorialRecursive() {
        if (num < 0) { cout << "Not defined for negative numbers.\n"; return; }
        cout << "Factorial (Recursive): " << factorialRecursive(num) << endl;
    }

    void calculateFactorialIterative() {
        if (num < 0) { cout << "Not defined for negative numbers.\n"; return; }
        long long fact = 1;
        for (int i = 2; i <= num; i++) fact *= i;
        cout << "Factorial (Iterative): " << fact << endl;
    }
}
```

```

void computeFactorsRecursive() {
    if (num <= 0) { cout << "Factors need a positive integer.\n"; return; }
    cout << "Factors (Recursive): ";
    factorsRecursive(num, 1);
    cout << endl;
}

void computeFactorsIterative() {
    if (num <= 0) { cout << "Factors need a positive integer.\n"; return; }
    cout << "Factors (Iterative): ";
    for (int i = 1; i <= num; i++) if (num % i == 0) cout << i << " ";
    cout << endl;
}
};

int main() {
    int n, choice;
    cout << "Enter a number: ";
    cin >> n;
    MathOperations op(n);

    while (true) {
        cout << "1.Factorial(Recursive) 2.Factorial(Iterative) "
            << "3.Factors(Recursive) 4.Factors(Iterative) 0.Change Number -1.Exit\n";
        cout << "Enter your choice: ";
        cin >> choice;

        switch (choice) {
            case 1: op.calculateFactorialRecursive(); break;
            case 2: op.calculateFactorialIterative(); break;
            case 3: op.computeFactorsRecursive(); break;
            case 4: op.computeFactorsIterative(); break;
            case 0: cout << "Enter new number: "; cin >> n; op = MathOperations(n); break;
            case -1: cout << "Exiting...\n"; return 0;
            default: cout << "Wrong choice.\n";
        }
    }
}

```

Output:

```
notamitgamer@fedora:~/bsc/semester_2/assignments on 7 main X!?  
~ > g++ assignment_13.cpp  
  
notamitgamer@fedora:~/bsc/semester_2/assignments on 7 main X!?  
~ > ./a.out  
Enter a number: 5  
1.Factorial(Recursive) 2.Factorial(Iterative)  
3.Factors(Recursive) 4.Factors(Iterative) 0.Change Number -1.Exit  
Enter your choice: 1  
Factorial (Recursive): 120  
1.Factorial(Recursive) 2.Factorial(Iterative)  
3.Factors(Recursive) 4.Factors(Iterative) 0.Change Number -1.Exit  
Enter your choice: 2  
Factorial (Iterative): 120  
1.Factorial(Recursive) 2.Factorial(Iterative)  
3.Factors(Recursive) 4.Factors(Iterative) 0.Change Number -1.Exit  
Enter your choice: 3  
Factors (Recursive): 1 5  
1.Factorial(Recursive) 2.Factorial(Iterative)  
3.Factors(Recursive) 4.Factors(Iterative) 0.Change Number -1.Exit  
Enter your choice: 4  
Factors (Iterative): 1 5  
1.Factorial(Recursive) 2.Factorial(Iterative)  
3.Factors(Recursive) 4.Factors(Iterative) 0.Change Number -1.Exit  
Enter your choice: 0  
Enter new number: 4  
1.Factorial(Recursive) 2.Factorial(Iterative)  
3.Factors(Recursive) 4.Factors(Iterative) 0.Change Number -1.Exit  
Enter your choice: 1  
Factorial (Recursive): 24  
1.Factorial(Recursive) 2.Factorial(Iterative)  
3.Factors(Recursive) 4.Factors(Iterative) 0.Change Number -1.Exit  
Enter your choice: 2  
Factorial (Iterative): 24  
1.Factorial(Recursive) 2.Factorial(Iterative)  
3.Factors(Recursive) 4.Factors(Iterative) 0.Change Number -1.Exit  
Enter your choice: 3  
Factors (Recursive): 1 2 4  
1.Factorial(Recursive) 2.Factorial(Iterative)  
3.Factors(Recursive) 4.Factors(Iterative) 0.Change Number -1.Exit  
Enter your choice: 4  
Factors (Iterative): 1 2 4  
1.Factorial(Recursive) 2.Factorial(Iterative)  
3.Factors(Recursive) 4.Factors(Iterative) 0.Change Number -1.Exit  
Enter your choice: -1  
Exiting...  
  
notamitgamer@fedora:~/bsc/semester_2/assignments on 7 main X!?  
~ > -
```

Discussion:

- **factorialRecursive()** breaks the problem down as `n * factorial(n-1)`, terminating at the base case `n <= 1`, building up the result through the call stack as it unwinds.
- **factorialIterative()** achieves the same result with a simple loop multiplying `fact` by each integer from 2 to `n`, avoiding recursive call overhead.
- **factorsRecursive()** checks divisibility for the current `divisor` and recurses with `divisor + 1` until it exceeds `n`, printing each valid divisor along the way.
- **factorsIterative()** performs the same check using a single `for` loop from 1 to `n`.

Assignment 15

Problem Statement:

Write a program to display the Fibonacci series using (i) recursion and (ii) iteration.

Problem Definition:

The **Fibonacci series** is a sequence where each term is the sum of the two preceding ones (0, 1, 1, 2, 3, 5, ...), defined recursively as $F(n) = F(n-1) + F(n-2)$ with base cases $F(0) = 0$ and $F(1) = 1$.

Algorithm:

Fibonacci Term using Recursion:

```
procedure fibRecursive(k)
begin
    if(k <= 0)
        return 0;
    end if
    if(k = 1)
        return 1;
    end if
    return fibRecursive(k - 1) + fibRecursive(k - 2);
end procedure
```

Fibonacci Series using Iteration:

```
procedure fibIterative(n)
begin
    if(n <= 0)
        return;
    end if
    first ← 0;
    second ← 1;
    for i ← 0 to n - 1 do
        if(i = 0)
            print first + " ";
        else if(i = 1)
            print second + " ";
        else
            nextTerm ← first + second;
            print nextTerm + " ";
            first ← second;
            second ← nextTerm;
        end if
    end for
end procedure
```

Source Code:

```
#include <iostream>
using namespace std;

class Fibonacci {
private:
    int n;

    int fibRecursive(int k) {
        if (k <= 0) return 0;
        if (k == 1) return 1;
        return fibRecursive(k - 1) + fibRecursive(k - 2);
    }

public:
    Fibonacci(int count) { n = count; }

    void displayRecursive() {
        cout << "Fibonacci (Recursive): ";
        for (int i = 0; i < n; i++) cout << fibRecursive(i) << " ";
        cout << endl;
    }

    void displayIterative() {
        cout << "Fibonacci (Iterative): ";
        if (n <= 0) { cout << endl; return; }
        int first = 0, second = 1;
        for (int i = 0; i < n; i++) {
            if (i == 0) cout << first << " ";
            else if (i == 1) cout << second << " ";
            else {
                int nextTerm = first + second;
                cout << nextTerm << " ";
                first = second;
                second = nextTerm;
            }
        }
        cout << endl;
    }
};

int main() {
    int n, choice;
    cout << "Enter number of terms: ";
    cin >> n;
    if (n < 0) { cout << "Invalid number of terms!\n"; return 0; }
```

```

Fibonacci fib(n);
while (true) {
    cout << "1.Recursive 2.Iterative -1.Exit\n";
    cout << "Enter your choice: ";
    cin >> choice;

    switch (choice) {
        case 1: fib.displayRecursive(); break;
        case 2: fib.displayIterative(); break;
        case -1: cout << "Exiting...\n"; return 0;
        default: cout << "Wrong choice.\n";
    }
}
}

```

Output:

```

notamitgamer@fedora:~/bsc/semester_2/assignments on  main X!?
~ > g++ assignment_14.cpp

notamitgamer@fedora:~/bsc/semester_2/assignments on  main X!?
~ > ./a.out
Enter number of terms: 10
1.Recursive 2.Iterative -1.Exit
Enter your choice: 1
Fibonacci (Recursive): 0 1 1 2 3 5 8 13 21 34
1.Recursive 2.Iterative -1.Exit
Enter your choice: 2
Fibonacci (Iterative): 0 1 1 2 3 5 8 13 21 34
1.Recursive 2.Iterative -1.Exit
Enter your choice: -1
Exiting...

notamitgamer@fedora:~/bsc/semester_2/assignments on  main X!?
~ > ./a.out
Enter number of terms: 15
1.Recursive 2.Iterative -1.Exit
Enter your choice: 1
Fibonacci (Recursive): 0 1 1 2 3 5 8 13 21 34 55 89 144 233 377
1.Recursive 2.Iterative -1.Exit
Enter your choice: 2
Fibonacci (Iterative): 0 1 1 2 3 5 8 13 21 34 55 89 144 233 377
1.Recursive 2.Iterative -1.Exit
Enter your choice: -1
Exiting...

notamitgamer@fedora:~/bsc/semester_2/assignments on  main X!?
~ > -

```

Discussion:

- **fibRecursive()** computes the k-th term by recursively summing the two preceding terms, with base cases returning 0 and 1 for $k \leq 0$ and $k = 1$ respectively — called once per term for the recursive display.
- **displayIterative()** builds the series in a single pass using two tracking variables, **first** and **second**, updating them each iteration to hold the last two computed terms, avoiding the exponential recomputation of the recursive approach.

Assignment 16

Problem Statement:

WAP to create a Binary Search Tree and include following operations in tree:

- i. Insertion (Recursive and Iterative Implementation)
- ii. Deletion by copying
- iii. Deletion by Merging
- iv. Search a no. in BST
- v. Display its preorder, postorder and inorder traversals Recursively
- vi. Display its preorder, postorder and inorder traversals Iteratively
- vii. Display its level-by-level traversals
- viii. Count the non-leaf nodes and leaf nodes
- ix. Display height of tree
- x. Create a mirror image of tree
- xi. Check whether two BSTs are equal or not

Problem Definition:

A **Binary Search Tree (BST)** is a binary tree in which every node's left subtree contains only values smaller than the node, and the right subtree contains only values larger. Key operations include **insertion**, **deletion** (by copying the inorder predecessor or by merging subtrees), **search**, and **traversals** (preorder, inorder, postorder, level order) to visit all nodes in different orders.

Algorithm:

Recursive Insertion:

```
procedure insertRecursive(root, val)
begin
    if(root = NULL)
        newNode ← create new node with val;
        return newNode;
    end if
    if(val < root.data)
        root.left ← insertRecursive(root.left, val);
    else if(val > root.data)
        root.right ← insertRecursive(root.right, val);
    else
        print "Duplicate value ignored";
    end if
    return root;
end procedure
```

Iterative Insertion:

```
procedure insertIterative(root, val)
begin
    newNode ← create new node with val;
```

```

        if(root = NULL)
            root ← newNode;
            return;
        end if
        current ← root;
        parent ← NULL;
        while(current ≠ NULL) do
            parent ← current;
            if(val < current.data)
                current ← current.left;
            else if(val > current.data)
                current ← current.right;
            else
                print "Duplicate value ignored";
                free newNode;
                return;
            end if
        end while
        if(val < parent.data)
            parent.left ← newNode;
        else
            parent.right ← newNode;
        end if
    end procedure

```

Search:

```

procedure search(root, key)
begin
    current ← root;
    while(current ≠ NULL) do
        if(current.data = key)
            return true;
        else if(key < current.data)
            current ← current.left;
        else
            current ← current.right;
        end if
    end while
    return false;
end procedure

```

Deletion by Copying:

```

procedure deleteByCopying(root, key)
begin
    if(root = NULL)
        return NULL;
    end if

```

```

end if
if(key < root.data)
    root.left ← deleteByCopying(root.left, key);
else if(key > root.data)
    root.right ← deleteByCopying(root.right, key);
else
    if(root.left = NULL)
        temp ← root.right;
        free root;
        return temp;
    else if(root.right = NULL)
        temp ← root.left;
        free root;
        return temp;
    end if
    pred ← root.left;
    while(pred.right ≠ NULL) do
        pred ← pred.right;
    end while
    root.data ← pred.data;
    root.left ← deleteByCopying(root.left, pred.data);
end if
return root;
end procedure

```

Deletion by Merging:

```

procedure deleteByMerging(root, key)
begin
    if(root = NULL)
        return NULL;
    end if
    if(key < root.data)
        root.left ← deleteByMerging(root.left, key);
        return root;
    else if(key > root.data)
        root.right ← deleteByMerging(root.right, key);
        return root;
    end if
    if(root.left = NULL)
        temp ← root.right;
        free root;
        return temp;
    else if(root.right = NULL)
        temp ← root.left;
        free root;
        return temp;
    end if
end if

```

```

        successor ← root.right;
        while(successor.left ≠ NULL) do
            successor ← successor.left;
        end while
        successor.left ← root.left;
        temp ← root.right;
        free root;
        return temp;
    end procedure

```

Preorder Traversal (Recursive):

```

procedure preorderRecursive(root)
begin
    if(root = NULL)
        return;
    end if
    print root.data + " ";
    call preorderRecursive(root.left);
    call preorderRecursive(root.right);
end procedure

```

Inorder Traversal (Recursive):

```

procedure inorderRecursive(root)
begin
    if(root = NULL)
        return;
    end if
    call inorderRecursive(root.left);
    print root.data + " ";
    call inorderRecursive(root.right);
end procedure

```

Postorder Traversal (Recursive):

```

procedure postorderRecursive(root)
begin
    if(root = NULL)
        return;
    end if
    call postorderRecursive(root.left);
    call postorderRecursive(root.right);
    print root.data + " ";
end procedure

```

Preorder Traversal (Iterative):

```
procedure preorderIterative(root)
begin
    if(root = NULL)
        return;
    end if
    initialize stack S;
    push(S, root);
    while(not isEmpty(S)) do
        current ← pop(S);
        print current.data + " ";
        if(current.right ≠ NULL)
            push(S, current.right);
        end if
        if(current.left ≠ NULL)
            push(S, current.left);
        end if
    end while
end procedure
```

Inorder Traversal (Iterative):

```
procedure inorderIterative(root)
begin
    if(root = NULL)
        return;
    end if
    initialize stack S;
    current ← root;
    while(current ≠ NULL or not isEmpty(S)) do
        while(current ≠ NULL) do
            push(S, current);
            current ← current.left;
        end while
        current ← pop(S);
        print current.data + " ";
        current ← current.right;
    end while
end procedure
```

Postorder Traversal (Iterative):

```
procedure postorderIterative(root)
begin
    if(root = NULL)
        return;
    end if
    initialize stacks S1, S2;
```

```

push(S1, root);
while(not isEmpty(S1)) do
    current ← pop(S1);
    push(S2, current);
    if(current.left ≠ NULL)
        push(S1, current.left);
    end if
    if(current.right ≠ NULL)
        push(S1, current.right);
    end if
end while
while(not isEmpty(S2)) do
    current ← pop(S2);
    print current.data + " ";
end while
end procedure

```

Level Order Traversal:

```

procedure levelOrder(root)
begin
    if(root = NULL)
        return;
    end if
    initialize queue Q;
    enqueue(Q, root);
    while(not isEmpty(Q)) do
        current ← dequeue(Q);
        print current.data + " ";
        if(current.left ≠ NULL)
            enqueue(Q, current.left);
        end if
        if(current.right ≠ NULL)
            enqueue(Q, current.right);
        end if
    end while
end procedure

```

Count Leaf Nodes:

```

procedure countLeaf(root)
begin
    if(root = NULL)
        return 0;
    end if
    if(root.left = NULL and root.right = NULL)
        return 1;
    end if

```

```
        return countLeaf(root.left) + countLeaf(root.right);
    end procedure
```

Count Non-Leaf Nodes:

```
procedure countNonLeaf(root)
begin
    if(root = NULL or (root.left = NULL and root.right = NULL))
        return 0;
    end if
    return 1 + countNonLeaf(root.left) + countNonLeaf(root.right);
end procedure
```

Calculate Height:

```
procedure height(root)
begin
    if(root = NULL)
        return -1;
    end if
    leftHeight ← height(root.left);
    rightHeight ← height(root.right);
    return 1 + max(leftHeight, rightHeight);
end procedure
```

Mirror Image:

```
procedure mirror(root)
begin
    if(root = NULL)
        return;
    end if
    temp ← root.left;
    root.left ← root.right;
    root.right ← temp;
    call mirror(root.left);
    call mirror(root.right);
end procedure
```

Check Trees Equal:

```
procedure areEqual(root1, root2)
begin
    if(root1 = NULL and root2 = NULL)
        return true;
    end if
    if(root1 = NULL or root2 = NULL)
        return false;
    end if
    if(areEqual(root1.left, root2.left) and areEqual(root1.right, root2.right))
        return true;
    end if
    return false;
end procedure
```

```

        end if
        if(root1.data ≠ root2.data)
            return false;
        end if
        return areEqual(root1.left, root2.left) and areEqual(root1.right, root2.right);
    end procedure

```

Source Code:

```

#include <iostream>
#include <stack>
#include <queue>
#include <algorithm>

using namespace std;

struct node {
    int data;
    node *left;
    node *right;

    node(int val) {
        data = val;
        left = nullptr;
        right = nullptr;
    }
};

class BST {
private:
    node *root;

    node* insertRecursive(node *current, int val, bool &inserted) {
        if (current == nullptr) {
            inserted = true;
            return new node(val);
        }
        if (val < current->data) current->left = insertRecursive(current->left, val,
inserted);
        else if (val > current->data) current->right = insertRecursive(current->right,
val, inserted);
        else { cout << "Duplicate value ignored.\n"; inserted = false; }
        return current;
    }

    void preorderRecursive(node *current) {
        if (current == nullptr) return;
        cout << current->data << " ";
    }
};

```

```

        preorderRecursive(current->left);
        preorderRecursive(current->right);
    }

    void inorderRecursive(node *current) {
        if (current == nullptr) return;
        inorderRecursive(current->left);
        cout << current->data << " ";
        inorderRecursive(current->right);
    }

    void postorderRecursive(node *current) {
        if (current == nullptr) return;
        postorderRecursive(current->left);
        postorderRecursive(current->right);
        cout << current->data << " ";
    }

    node* deleteByCopying(node *current, int key) {
        if (current == nullptr) return nullptr;
        if (key < current->data) {
            current->left = deleteByCopying(current->left, key);
        } else if (key > current->data) {
            current->right = deleteByCopying(current->right, key);
        } else {
            if (current->left == nullptr) { node *temp = current->right; delete current;
return temp; }
            if (current->right == nullptr) { node *temp = current->left; delete current;
return temp; }
            node *pred = current->left;
            while (pred->right != nullptr) pred = pred->right;
            current->data = pred->data;
            current->left = deleteByCopying(current->left, pred->data);
        }
        return current;
    }

    node* deleteByMerging(node *current, int key) {
        if (current == nullptr) return nullptr;
        if (key < current->data) { current->left = deleteByMerging(current->left, key);
return current; }
        if (key > current->data) { current->right = deleteByMerging(current->right, key);
return current; }
        if (current->left == nullptr) { node *temp = current->right; delete current;
return temp; }
        if (current->right == nullptr) { node *temp = current->left; delete current;
return temp; }
        node *successor = current->right;

```

```

        while (successor->left != nullptr) successor = successor->left;
        successor->left = current->left;
        node *temp = current->right;
        delete current;
        return temp;
    }

    int countLeaf(node *current) {
        if (current == nullptr) return 0;
        if (current->left == nullptr && current->right == nullptr) return 1;
        return countLeaf(current->left) + countLeaf(current->right);
    }

    int countNonLeaf(node *current) {
        if (current == nullptr) return 0;
        if (current->left == nullptr && current->right == nullptr) return 0;
        return 1 + countNonLeaf(current->left) + countNonLeaf(current->right);
    }

    int height(node *current) {
        if (current == nullptr) return -1;
        return 1 + max(height(current->left), height(current->right));
    }

    void mirror(node *current) {
        if (current == nullptr) return;
        node *temp = current->left;
        current->left = current->right;
        current->right = temp;
        mirror(current->left);
        mirror(current->right);
    }

    bool areEqual(node *root1, node *root2) {
        if (root1 == nullptr && root2 == nullptr) return true;
        if (root1 == nullptr || root2 == nullptr) return false;
        if (root1->data != root2->data) return false;
        return areEqual(root1->left, root2->left) && areEqual(root1->right, root2->right);
    }

    void destroy(node *current) {
        if (current == nullptr) return;
        destroy(current->left);
        destroy(current->right);
        delete current;
    }

public:

```

```

BST() { root = nullptr; }

void insertRecursive(int val) {
    bool inserted = false;
    root = insertRecursive(root, val, inserted);
}

void insertIterative(int val) {
    node *newNode = new node(val);
    if (root == nullptr) { root = newNode; return; }
    node *current = root;
    node *parent = nullptr;
    while (current != nullptr) {
        parent = current;
        if (val < current->data) current = current->left;
        else if (val > current->data) current = current->right;
        else { cout << "Duplicate value ignored.\n"; delete newNode; return; }
    }
    if (val < parent->data) parent->left = newNode;
    else parent->right = newNode;
}

void deleteCopying(int key) {
    if (!search(key)) { cout << "Element not found in BST.\n"; return; }
    root = deleteByCopying(root, key);
}

void deleteMerging(int key) {
    if (!search(key)) { cout << "Element not found in BST.\n"; return; }
    root = deleteByMerging(root, key);
}

bool search(int key) {
    node *current = root;
    while (current != nullptr) {
        if (current->data == key) return true;
        current = (key < current->data) ? current->left : current->right;
    }
    return false;
}

void preorderRecursive() { preorderRecursive(root); cout << endl; }
void inorderRecursive() { inorderRecursive(root); cout << endl; }
void postorderRecursive() { postorderRecursive(root); cout << endl; }

void preorderIterative() {
    if (root == nullptr) return;
    stack<node*> s;
}

```

```

        s.push(root);
        while (!s.empty()) {
            node *current = s.top(); s.pop();
            cout << current->data << " ";
            if (current->right != nullptr) s.push(current->right);
            if (current->left != nullptr) s.push(current->left);
        }
        cout << endl;
    }

    void inorderIterative() {
        stack<node*> s;
        node *current = root;
        while (current != nullptr || !s.empty()) {
            while (current != nullptr) { s.push(current); current = current->left; }
            current = s.top(); s.pop();
            cout << current->data << " ";
            current = current->right;
        }
        cout << endl;
    }

    void postorderIterative() {
        if (root == nullptr) return;
        stack<node*> s1, s2;
        s1.push(root);
        while (!s1.empty()) {
            node *current = s1.top(); s1.pop();
            s2.push(current);
            if (current->left != nullptr) s1.push(current->left);
            if (current->right != nullptr) s1.push(current->right);
        }
        while (!s2.empty()) { cout << s2.top()->data << " "; s2.pop(); }
        cout << endl;
    }

    void levelOrder() {
        if (root == nullptr) return;
        queue<node*> q;
        q.push(root);
        while (!q.empty()) {
            node *current = q.front(); q.pop();
            cout << current->data << " ";
            if (current->left != nullptr) q.push(current->left);
            if (current->right != nullptr) q.push(current->right);
        }
        cout << endl;
    }
}

```

```

void countNodes() {
    cout << "Leaf Nodes: " << countLeaf(root) << endl;
    cout << "Non-Leaf Nodes: " << countNonLeaf(root) << endl;
}

void displayHeight() { cout << "Height of Tree: " << height(root) << endl; }

void createMirror() { mirror(root); }

bool isEqual(BST &other) { return areEqual(root, other.root); }

~BST() { destroy(root); }
};

int main() {
    BST tree;
    int choice, value;

    while (true) {
        cout << "1.InsertRec 2.InsertIter 3.DeleteCopy 4.DeleteMerge 5.Search\n";
        cout << "6.PreorderRec 7.InorderRec 8.PostorderRec 9.PreorderIter
10.InorderIter\n";
        cout << "11.PostorderIter 12.LevelOrder 13.CountNodes 14.Height 15.Mirror
16.CheckEqual 0.Exit\n";
        cout << "Enter your choice: ";
        cin >> choice;

        switch (choice) {
            case 1: cout << "Enter value: "; cin >> value; tree.insertRecursive(value);
break;
            case 2: cout << "Enter value: "; cin >> value; tree.insertIterative(value);
break;
            case 3: cout << "Enter value to delete: "; cin >> value;
tree.deleteCopying(value); break;
            case 4: cout << "Enter value to delete: "; cin >> value;
tree.deleteMerging(value); break;
            case 5:
                cout << "Enter value to search: "; cin >> value;
                cout << (tree.search(value) ? "Found\n" : "Not found\n");
                break;
            case 6: tree.preorderRecursive(); break;
            case 7: tree.inorderRecursive(); break;
            case 8: tree.postorderRecursive(); break;
            case 9: tree.preorderIterative(); break;
            case 10: tree.inorderIterative(); break;
            case 11: tree.postorderIterative(); break;
            case 12: tree.levelOrder(); break;

```

```

        case 13: tree.countNodes(); break;
        case 14: tree.displayHeight(); break;
        case 15: tree.createMirror(); break;
        case 16: {
            BST tree2;
            int n;
            cout << "Enter number of nodes for second BST: "; cin >> n;
            for (int i = 0; i < n; i++) { cin >> value; tree2.insertIterative(value);
}
            cout << (tree.isEqual(tree2) ? "Both BSTs are equal.\n" : "BSTs are not
equal.\n");
            break;
        }
        case 0: return 0;
        default: cout << "Wrong choice.\n";
    }
}
}

```

Output:

```
notamitgamer@fedora:~/bsc/semester_2/assignments on ? main X!?  
~ ) g++ assignment_15.cpp  
  
notamitgamer@fedora:~/bsc/semester_2/assignments on ? main X!?  
~ ) ./a.out  
1.InsertRec 2.InsertIter 3.DeleteCopy 4.DeleteMerge 5.Search  
6.PreorderRec 7.InorderRec 8.PostorderRec 9.PreorderIter 10.InorderIter  
11.PostorderIter 12.LevelOrder 13.CountNodes 14.Height 15.Mirror 16.CheckEqual 0.Exit  
Enter your choice: 1  
Enter value: 25  
1.InsertRec 2.InsertIter 3.DeleteCopy 4.DeleteMerge 5.Search  
6.PreorderRec 7.InorderRec 8.PostorderRec 9.PreorderIter 10.InorderIter  
11.PostorderIter 12.LevelOrder 13.CountNodes 14.Height 15.Mirror 16.CheckEqual 0.Exit  
Enter your choice: 1  
Enter value: 15  
1.InsertRec 2.InsertIter 3.DeleteCopy 4.DeleteMerge 5.Search  
6.PreorderRec 7.InorderRec 8.PostorderRec 9.PreorderIter 10.InorderIter  
11.PostorderIter 12.LevelOrder 13.CountNodes 14.Height 15.Mirror 16.CheckEqual 0.Exit  
Enter your choice: 2  
Enter value: 35  
1.InsertRec 2.InsertIter 3.DeleteCopy 4.DeleteMerge 5.Search  
6.PreorderRec 7.InorderRec 8.PostorderRec 9.PreorderIter 10.InorderIter  
11.PostorderIter 12.LevelOrder 13.CountNodes 14.Height 15.Mirror 16.CheckEqual 0.Exit  
Enter your choice: 1  
Enter value: 10  
1.InsertRec 2.InsertIter 3.DeleteCopy 4.DeleteMerge 5.Search  
6.PreorderRec 7.InorderRec 8.PostorderRec 9.PreorderIter 10.InorderIter  
11.PostorderIter 12.LevelOrder 13.CountNodes 14.Height 15.Mirror 16.CheckEqual 0.Exit  
Enter your choice: 2  
Enter value: 20  
1.InsertRec 2.InsertIter 3.DeleteCopy 4.DeleteMerge 5.Search  
6.PreorderRec 7.InorderRec 8.PostorderRec 9.PreorderIter 10.InorderIter  
11.PostorderIter 12.LevelOrder 13.CountNodes 14.Height 15.Mirror 16.CheckEqual 0.Exit  
Enter your choice: 1  
Enter value: 30  
1.InsertRec 2.InsertIter 3.DeleteCopy 4.DeleteMerge 5.Search  
6.PreorderRec 7.InorderRec 8.PostorderRec 9.PreorderIter 10.InorderIter  
11.PostorderIter 12.LevelOrder 13.CountNodes 14.Height 15.Mirror 16.CheckEqual 0.Exit  
Enter your choice: 2  
Enter value: 40  
1.InsertRec 2.InsertIter 3.DeleteCopy 4.DeleteMerge 5.Search  
6.PreorderRec 7.InorderRec 8.PostorderRec 9.PreorderIter 10.InorderIter  
11.PostorderIter 12.LevelOrder 13.CountNodes 14.Height 15.Mirror 16.CheckEqual 0.Exit  
Enter your choice: 5  
Enter value to search: 20  
Found  
1.InsertRec 2.InsertIter 3.DeleteCopy 4.DeleteMerge 5.Search  
6.PreorderRec 7.InorderRec 8.PostorderRec 9.PreorderIter 10.InorderIter  
11.PostorderIter 12.LevelOrder 13.CountNodes 14.Height 15.Mirror 16.CheckEqual 0.Exit  
Enter your choice: 5  
Enter value to search: 99  
Not found
```

Continue to next page...

```

1.InsertRec 2.InsertIter 3.DeleteCopy 4.DeleteMerge 5.Search
6.PreorderRec 7.InorderRec 8.PostorderRec 9.PreorderIter 10.InorderIter
11.PostorderIter 12.LevelOrder 13.CountNodes 14.Height 15.Mirror 16.CheckEqual 0.Exit
Enter your choice: 6
25 15 10 20 35 30 40
1.InsertRec 2.InsertIter 3.DeleteCopy 4.DeleteMerge 5.Search
6.PreorderRec 7.InorderRec 8.PostorderRec 9.PreorderIter 10.InorderIter
11.PostorderIter 12.LevelOrder 13.CountNodes 14.Height 15.Mirror 16.CheckEqual 0.Exit
Enter your choice: 7
10 15 20 25 30 35 40
1.InsertRec 2.InsertIter 3.DeleteCopy 4.DeleteMerge 5.Search
6.PreorderRec 7.InorderRec 8.PostorderRec 9.PreorderIter 10.InorderIter
11.PostorderIter 12.LevelOrder 13.CountNodes 14.Height 15.Mirror 16.CheckEqual 0.Exit
Enter your choice: 8
10 20 15 30 40 35 25
1.InsertRec 2.InsertIter 3.DeleteCopy 4.DeleteMerge 5.Search
6.PreorderRec 7.InorderRec 8.PostorderRec 9.PreorderIter 10.InorderIter
11.PostorderIter 12.LevelOrder 13.CountNodes 14.Height 15.Mirror 16.CheckEqual 0.Exit
Enter your choice: 9
25 15 10 20 35 30 40
1.InsertRec 2.InsertIter 3.DeleteCopy 4.DeleteMerge 5.Search
6.PreorderRec 7.InorderRec 8.PostorderRec 9.PreorderIter 10.InorderIter
11.PostorderIter 12.LevelOrder 13.CountNodes 14.Height 15.Mirror 16.CheckEqual 0.Exit
Enter your choice: 10
10 15 20 25 30 35 40
1.InsertRec 2.InsertIter 3.DeleteCopy 4.DeleteMerge 5.Search
6.PreorderRec 7.InorderRec 8.PostorderRec 9.PreorderIter 10.InorderIter
11.PostorderIter 12.LevelOrder 13.CountNodes 14.Height 15.Mirror 16.CheckEqual 0.Exit
Enter your choice: 11
10 20 15 30 40 35 25
1.InsertRec 2.InsertIter 3.DeleteCopy 4.DeleteMerge 5.Search
6.PreorderRec 7.InorderRec 8.PostorderRec 9.PreorderIter 10.InorderIter
11.PostorderIter 12.LevelOrder 13.CountNodes 14.Height 15.Mirror 16.CheckEqual 0.Exit
Enter your choice: 12
25 15 35 10 20 30 40
1.InsertRec 2.InsertIter 3.DeleteCopy 4.DeleteMerge 5.Search
6.PreorderRec 7.InorderRec 8.PostorderRec 9.PreorderIter 10.InorderIter
11.PostorderIter 12.LevelOrder 13.CountNodes 14.Height 15.Mirror 16.CheckEqual 0.Exit
Enter your choice: 13
Leaf Nodes: 4
Non-Leaf Nodes: 3
1.InsertRec 2.InsertIter 3.DeleteCopy 4.DeleteMerge 5.Search
6.PreorderRec 7.InorderRec 8.PostorderRec 9.PreorderIter 10.InorderIter
11.PostorderIter 12.LevelOrder 13.CountNodes 14.Height 15.Mirror 16.CheckEqual 0.Exit
Enter your choice: 14
Height of Tree: 2
1.InsertRec 2.InsertIter 3.DeleteCopy 4.DeleteMerge 5.Search
6.PreorderRec 7.InorderRec 8.PostorderRec 9.PreorderIter 10.InorderIter
11.PostorderIter 12.LevelOrder 13.CountNodes 14.Height 15.Mirror 16.CheckEqual 0.Exit
Enter your choice: 16
Enter number of nodes for second BST: 3
10 20 30
BSTs are not equal.

```

Continue to next page...

```

1.InsertRec 2.InsertIter 3.DeleteCopy 4.DeleteMerge 5.Search
6.PreorderRec 7.InorderRec 8.PostorderRec 9.PreorderIter 10.InorderIter
11.PostorderIter 12.LevelOrder 13.CountNodes 14.Height 15.Mirror 16.CheckEqual 0.Exit
Enter your choice: 3
Enter value to delete: 15
1.InsertRec 2.InsertIter 3.DeleteCopy 4.DeleteMerge 5.Search
6.PreorderRec 7.InorderRec 8.PostorderRec 9.PreorderIter 10.InorderIter
11.PostorderIter 12.LevelOrder 13.CountNodes 14.Height 15.Mirror 16.CheckEqual 0.Exit
Enter your choice: 12
25 10 35 20 30 40
1.InsertRec 2.InsertIter 3.DeleteCopy 4.DeleteMerge 5.Search
6.PreorderRec 7.InorderRec 8.PostorderRec 9.PreorderIter 10.InorderIter
11.PostorderIter 12.LevelOrder 13.CountNodes 14.Height 15.Mirror 16.CheckEqual 0.Exit
Enter your choice: 7
10 20 25 30 35 40
1.InsertRec 2.InsertIter 3.DeleteCopy 4.DeleteMerge 5.Search
6.PreorderRec 7.InorderRec 8.PostorderRec 9.PreorderIter 10.InorderIter
11.PostorderIter 12.LevelOrder 13.CountNodes 14.Height 15.Mirror 16.CheckEqual 0.Exit
Enter your choice: 4
Enter value to delete: 35
1.InsertRec 2.InsertIter 3.DeleteCopy 4.DeleteMerge 5.Search
6.PreorderRec 7.InorderRec 8.PostorderRec 9.PreorderIter 10.InorderIter
11.PostorderIter 12.LevelOrder 13.CountNodes 14.Height 15.Mirror 16.CheckEqual 0.Exit
Enter your choice: 12
25 10 40 20 30
1.InsertRec 2.InsertIter 3.DeleteCopy 4.DeleteMerge 5.Search
6.PreorderRec 7.InorderRec 8.PostorderRec 9.PreorderIter 10.InorderIter
11.PostorderIter 12.LevelOrder 13.CountNodes 14.Height 15.Mirror 16.CheckEqual 0.Exit
Enter your choice: 7
10 20 25 30 40
1.InsertRec 2.InsertIter 3.DeleteCopy 4.DeleteMerge 5.Search
6.PreorderRec 7.InorderRec 8.PostorderRec 9.PreorderIter 10.InorderIter
11.PostorderIter 12.LevelOrder 13.CountNodes 14.Height 15.Mirror 16.CheckEqual 0.Exit
Enter your choice: 15
1.InsertRec 2.InsertIter 3.DeleteCopy 4.DeleteMerge 5.Search
6.PreorderRec 7.InorderRec 8.PostorderRec 9.PreorderIter 10.InorderIter
11.PostorderIter 12.LevelOrder 13.CountNodes 14.Height 15.Mirror 16.CheckEqual 0.Exit
Enter your choice: 7
40 30 25 20 10
1.InsertRec 2.InsertIter 3.DeleteCopy 4.DeleteMerge 5.Search
6.PreorderRec 7.InorderRec 8.PostorderRec 9.PreorderIter 10.InorderIter
11.PostorderIter 12.LevelOrder 13.CountNodes 14.Height 15.Mirror 16.CheckEqual 0.Exit
Enter your choice: 12
25 40 10 30 20
1.InsertRec 2.InsertIter 3.DeleteCopy 4.DeleteMerge 5.Search
6.PreorderRec 7.InorderRec 8.PostorderRec 9.PreorderIter 10.InorderIter
11.PostorderIter 12.LevelOrder 13.CountNodes 14.Height 15.Mirror 16.CheckEqual 0.Exit
Enter your choice: 0

```

```

notamitgamer@fedora:~/bsc/semester_2/assignments on 7 main X!?
~ )

```

Discussion:

- **insertRecursive()** recursively finds the correct NULL position by comparing with each node's data and attaches the new node there.
- **insertIterative()** walks down the tree using a `current / parent` pointer pair until it finds the correct empty spot, then links the new node.
- **search()** iteratively compares the key against each node's data, moving left or right until found or the tree is exhausted.
- **deleteByCopying()** locates the node, then replaces its data with the inorder predecessor's data and recursively deletes that predecessor.
- **deleteByMerging()** locates the node and, if it has two children, attaches its left subtree under the leftmost node of its right subtree, then removes the node.
- **preorderRecursive()** / **inorderRecursive()** / **postorderRecursive()** visit root-left-right, left-root-right, and left-right-root respectively using recursive calls.
- **preorderIterative()** uses a stack, pushing right then left children so left is processed first.
- **inorderIterative()** pushes left children onto a stack until NULL, then pops, prints, and moves right.
- **postorderIterative()** uses two stacks: the first collects nodes in a modified preorder, the second reverses that order for postorder output.
- **levelOrder()** uses a queue to visit nodes breadth-first, enqueueing children as each node is dequeued.
- **countLeaf()** / **countNonLeaf()** recursively count nodes with no children versus nodes with at least one child.
- **height()** recursively computes `1 + max(leftHeight, rightHeight)`, with an empty tree returning -1.
- **mirror()** recursively swaps the left and right children of every node.
- **areEqual()** recursively compares two trees node by node, checking both structure and data.

Assignment 17

Problem Statement:

WAP to convert the Sparse Matrix into non-zero form and vice-versa.

Problem Definition:

A **Sparse Matrix** is a matrix in which most elements are zero. Storing it in standard 2D form wastes memory, so it is instead represented in **3-tuple (triplet) form**, storing only the row, column, and value of each non-zero element, along with the matrix's total rows, columns, and non-zero count in the first row.

Algorithm:

Convert Standard 2D Matrix to 3-Tuple Representation:

```
procedure convertToTriplet(matrix, rows, cols)
begin
    count ← 0;
    for i ← 0 to rows - 1 do
        for j ← 0 to cols - 1 do
            if(matrix[i][j] ≠ 0)
                count ← count + 1;
            end if
        end for
    end for

    allocate triplet of size (count + 1) x 3;
    triplet[0][0] ← rows;
    triplet[0][1] ← cols;
    triplet[0][2] ← count;

    k ← 1;
    for i ← 0 to rows - 1 do
        for j ← 0 to cols - 1 do
            if(matrix[i][j] ≠ 0)
                triplet[k][0] ← i;
                triplet[k][1] ← j;
                triplet[k][2] ← matrix[i][j];
                k ← k + 1;
            end if
        end for
    end for
end procedure
```

Convert 3-Tuple Representation to Standard 2D Matrix:

```
procedure convertToStandard(triplet)
begin
```

```

rows ← triplet[0][0];
cols ← triplet[0][1];
count ← triplet[0][2];

allocate matrix of size rows x cols;
for i ← 0 to rows - 1 do
    for j ← 0 to cols - 1 do
        matrix[i][j] ← 0;
    end for
end for

for i ← 1 to count do
    r ← triplet[i][0];
    c ← triplet[i][1];
    val ← triplet[i][2];
    if(r >= 0 and r < rows and c >= 0 and c < cols)
        matrix[r][c] ← val;
    end if
end for
end procedure

```

Source Code:

```

#include <iostream>
using namespace std;

class SparseMatrix {
private:
    int rows;
    int cols;
    int **standardMatrix;
    int **triplet;
    int nonZeroCount;

    void allocateStandard(int r, int c) {
        rows = r;
        cols = c;
        standardMatrix = new int*[rows];
        for (int i = 0; i < rows; i++) {
            standardMatrix[i] = new int[cols];
            for (int j = 0; j < cols; j++) standardMatrix[i][j] = 0;
        }
    }

    void freeStandard() {
        if (standardMatrix != nullptr) {
            for (int i = 0; i < rows; i++) delete[] standardMatrix[i];
            delete[] standardMatrix;
        }
    }
};

```

```

        standardMatrix = nullptr;
    }
}

void freeTriplet() {
    if (triplet != nullptr) {
        for (int i = 0; i <= nonZeroCount; i++) delete[] triplet[i];
        delete[] triplet;
        triplet = nullptr;
    }
}

public:
    SparseMatrix() {
        rows = 0; cols = 0; nonZeroCount = 0;
        standardMatrix = nullptr;
        triplet = nullptr;
    }

    void inputStandardMatrix() {
        freeStandard();
        freeTriplet();
        cout << "Enter number of rows: "; cin >> rows;
        cout << "Enter number of columns: "; cin >> cols;
        if (rows <= 0 || cols <= 0) { cout << "Invalid matrix dimensions!\n"; rows = cols
= 0; return; }
        allocateStandard(rows, cols);
        cout << "Enter matrix elements (" << rows << " x " << cols << "):\n";
        for (int i = 0; i < rows; i++)
            for (int j = 0; j < cols; j++) cin >> standardMatrix[i][j];
    }

    void convertToTriplet() {
        if (standardMatrix == nullptr) { cout << "No standard matrix available!\n";
return; }
        freeTriplet();
        nonZeroCount = 0;
        for (int i = 0; i < rows; i++)
            for (int j = 0; j < cols; j++)
                if (standardMatrix[i][j] != 0) nonZeroCount++;

        triplet = new int*[nonZeroCount + 1];
        for (int i = 0; i <= nonZeroCount; i++) triplet[i] = new int[3];

        triplet[0][0] = rows;
        triplet[0][1] = cols;
        triplet[0][2] = nonZeroCount;
    }
}

```

```

        int k = 1;
        for (int i = 0; i < rows; i++)
            for (int j = 0; j < cols; j++)
                if (standardMatrix[i][j] != 0) {
                    triplet[k][0] = i;
                    triplet[k][1] = j;
                    triplet[k][2] = standardMatrix[i][j];
                    k++;
                }
    }

    void inputTripletForm() {
        freeStandard();
        freeTriplet();
        int r, c, count;
        cout << "Enter total rows: "; cin >> r;
        cout << "Enter total columns: "; cin >> c;
        cout << "Enter number of non-zero elements: "; cin >> count;
        if (r <= 0 || c <= 0 || count < 0 || count > r * c) { cout << "Invalid input
parameters!\n"; return; }

        rows = r; cols = c; nonZeroCount = count;
        triplet = new int*[nonZeroCount + 1];
        for (int i = 0; i <= nonZeroCount; i++) triplet[i] = new int[3];

        triplet[0][0] = rows;
        triplet[0][1] = cols;
        triplet[0][2] = nonZeroCount;

        cout << "Enter (row, col, value) for each non-zero element:\n";
        for (int i = 1; i <= nonZeroCount; i++) cin >> triplet[i][0] >> triplet[i][1] >>
triplet[i][2];
    }

    void convertToStandard() {
        if (triplet == nullptr) { cout << "No 3-tuple representation found!\n"; return; }
        freeStandard();
        rows = triplet[0][0];
        cols = triplet[0][1];
        nonZeroCount = triplet[0][2];
        allocateStandard(rows, cols);

        for (int i = 1; i <= nonZeroCount; i++) {
            int r = triplet[i][0];
            int c = triplet[i][1];
            int val = triplet[i][2];
            if (r >= 0 && r < rows && c >= 0 && c < cols) standardMatrix[r][c] = val;
        }
    }

```

```

    }

    void displayStandard() {
        if (standardMatrix == nullptr) { cout << "Standard matrix is empty!\n"; return; }
        for (int i = 0; i < rows; i++) {
            for (int j = 0; j < cols; j++) cout << standardMatrix[i][j] << " ";
            cout << endl;
        }
    }

    void displayTriplet() {
        if (triplet == nullptr) { cout << "3-Tuple representation is empty!\n"; return; }
        for (int i = 0; i <= nonZeroCount; i++)
            cout << triplet[i][0] << " " << triplet[i][1] << " " << triplet[i][2] << endl;
    }

    ~SparseMatrix() {
        freeStandard();
        freeTriplet();
    }
};

int main() {
    SparseMatrix sm;
    int choice;

    while (true) {
        cout << "1.InputMatrix 2.ToTriplet 3.DisplayTriplet \n4.InputTriplet 5.ToStandard\n6.DisplayMatrix -1.Exit\n";
        cout << "Enter your choice: ";
        cin >> choice;

        switch (choice) {
            case 1: sm.inputStandardMatrix(); break;
            case 2: sm.convertToTriplet(); break;
            case 3: sm.displayTriplet(); break;
            case 4: sm.inputTripletForm(); break;
            case 5: sm.convertToStandard(); break;
            case 6: sm.displayStandard(); break;
            case -1: cout << "Exiting...\n"; return 0;
            default: cout << "Wrong choice.\n";
        }
    }
}

```

Output:

```
notamitgamer@fedora:~/bsc/semester_2/assignments on  main X!?  
~ > g++ assignment_16.cpp  
  
notamitgamer@fedora:~/bsc/semester_2/assignments on  main X!?  
~ > ./a.out  
1.InputMatrix 2.ToTriplet 3.DisplayTriplet  
4.InputTriplet 5.ToStandard 6.DisplayMatrix -1.Exit  
Enter your choice: 1  
Enter number of rows: 4  
Enter number of columns: 4  
Enter matrix elements (4 x 4):  
0 0 5 0  
0 8 0 0  
0 0 0 0  
2 0 0 0  
1.InputMatrix 2.ToTriplet 3.DisplayTriplet  
4.InputTriplet 5.ToStandard 6.DisplayMatrix -1.Exit  
Enter your choice: 2  
1.InputMatrix 2.ToTriplet 3.DisplayTriplet  
4.InputTriplet 5.ToStandard 6.DisplayMatrix -1.Exit  
Enter your choice: 3  
4 4 3  
0 2 5  
1 1 8  
3 0 2  
1.InputMatrix 2.ToTriplet 3.DisplayTriplet  
4.InputTriplet 5.ToStandard 6.DisplayMatrix -1.Exit  
Enter your choice: 6  
0 0 5 0  
0 8 0 0  
0 0 0 0  
2 0 0 0  
1.InputMatrix 2.ToTriplet 3.DisplayTriplet  
4.InputTriplet 5.ToStandard 6.DisplayMatrix -1.Exit  
Enter your choice: 4  
Enter total rows: 3  
Enter total columns: 3  
Enter number of non-zero elements: 2  
Enter (row, col, value) for each non-zero element:  
0 1 9  
2 2 4  
1.InputMatrix 2.ToTriplet 3.DisplayTriplet  
4.InputTriplet 5.ToStandard 6.DisplayMatrix -1.Exit  
Enter your choice: 3  
3 3 2  
0 1 9  
2 2 4  
  
1.InputMatrix 2.ToTriplet 3.DisplayTriplet  
4.InputTriplet 5.ToStandard 6.DisplayMatrix -1.Exit  
Enter your choice: 5  
1.InputMatrix 2.ToTriplet 3.DisplayTriplet  
4.InputTriplet 5.ToStandard 6.DisplayMatrix -1.Exit  
Enter your choice: 6  
0 9 0  
0 0 0  
0 0 4  
1.InputMatrix 2.ToTriplet 3.DisplayTriplet  
4.InputTriplet 5.ToStandard 6.DisplayMatrix -1.Exit  
Enter your choice: -1  
Exiting...
```

```
notamitgamer@fedora:~/bsc/semester_2/assignments on  main X!?  
~ >
```

Discussion:

- **convertToTriplet()** scans the 2D matrix to count non-zero elements, allocates a triplet array of that size, and fills it with the row, column, and value of each non-zero entry, with row 0 storing the matrix dimensions and non-zero count.
- **convertToStandard()** reads the dimensions and count from row 0 of the triplet array, allocates a zero-filled 2D matrix, and places each stored value at its recorded row and column.

Assignment 18

Problem Statement:

WAP to reverse the order of the elements in the stack using additional stack.

Problem Definition:

Reversing a **Stack** means making the bottom element the new top and vice versa. This can be achieved using two **auxiliary stacks**: elements are popped off the original stack and pushed twice across two helper stacks, which reverses the order twice and restores the elements to the original stack in reversed order.

Algorithm:

Reverse Stack Using Additional Stacks:

```
procedure reverseUsingAdditionalStacks(S, capacity)
begin
    if(isEmpty(S))
        print "Stack is empty! Nothing to reverse.";
        return;
    end if

    initialize stack S1 of capacity;
    initialize stack S2 of capacity;

    while(not isEmpty(S)) do
        val ← pop(S);
        push(S1, val);
    end while

    while(not isEmpty(S1)) do
        val ← pop(S1);
        push(S2, val);
    end while

    while(not isEmpty(S2)) do
        val ← pop(S2);
        push(S, val);
    end while
end procedure
```

Source Code:

```
#include <iostream>
using namespace std;

class Stack {
private:
```

```

int *arr;
int top;
int capacity;

public:
    Stack(int size) {
        capacity = size;
        arr = new int[capacity];
        top = -1;
    }

    Stack(const Stack&) = delete;
    Stack& operator=(const Stack&) = delete;

    bool isFull() const { return (top == capacity - 1); }
    bool isEmpty() const { return (top == -1); }

    void push(int val) {
        if (isFull()) { cout << "Stack Overflow!\n"; return; }
        arr[++top] = val;
    }

    int pop() {
        if (isEmpty()) { cout << "Stack Underflow!\n"; return -1; }
        return arr[top--];
    }

    void display() const {
        if (isEmpty()) { cout << "Stack is empty!\n"; return; }
        for (int i = top; i >= 0; i--) cout << arr[i] << " ";
        cout << endl;
    }

    void reverseUsingAdditionalStacks() {
        if (isEmpty()) { cout << "Stack is empty! Nothing to reverse.\n"; return; }

        Stack s1(capacity);
        Stack s2(capacity);

        while (!isEmpty()) s1.push(pop());
        while (!s1.isEmpty()) s2.push(s1.pop());
        while (!s2.isEmpty()) push(s2.pop());
    }

    ~Stack() { delete[] arr; }
};

int main() {

```

```

int size, choice, val;
cout << "Enter the capacity of the stack: ";
cin >> size;
if (size <= 0) { cout << "Invalid stack size!\n"; return 0; }

Stack s(size);

while (true) {
    cout << "1.push 2.pop 3.reverse 0.display -1.Exit\n";
    cout << "Enter your choice: ";
    cin >> choice;

    switch (choice) {
        case 1:
            cout << "Enter value: ";
            cin >> val;
            s.push(val);
            break;
        case 2: {
            int v = s.pop();
            if (v != -1) cout << "Popped element: " << v << endl;
            break;
        }
        case 3:
            s.reverseUsingAdditionalStacks();
            cout << "Stack reversed.\n"; // Added feedback
            break;
        case 0:
            s.display();
            break;
        case -1:
            cout << "Exiting...\n";
            return 0;
        default:
            cout << "Wrong choice.\n";
    }
}
}

```

Output:

```
notamitgamer@fedora:~/bsc/semester_2/assignments on ? main X!?  
~ > g++ assignment_17.cpp  
  
notamitgamer@fedora:~/bsc/semester_2/assignments on ? main X!?  
~ > ./a.out  
Enter the capacity of the stack: 4  
1.push 2.pop 3.reverse 0.display -1.Exit  
Enter your choice: 1  
Enter value: 10  
1.push 2.pop 3.reverse 0.display -1.Exit  
Enter your choice: 1  
Enter value: 20  
1.push 2.pop 3.reverse 0.display -1.Exit  
Enter your choice: 1  
Enter value: 30  
1.push 2.pop 3.reverse 0.display -1.Exit  
Enter your choice: 1  
Enter value: 40  
1.push 2.pop 3.reverse 0.display -1.Exit  
Enter your choice: 1  
Enter value: 50  
Stack Overflow!  
1.push 2.pop 3.reverse 0.display -1.Exit  
Enter your choice: 0  
40 30 20 10  
1.push 2.pop 3.reverse 0.display -1.Exit  
Enter your choice: 3  
Stack reversed.  
1.push 2.pop 3.reverse 0.display -1.Exit  
Enter your choice: 0  
10 20 30 40  
1.push 2.pop 3.reverse 0.display -1.Exit  
Enter your choice: 2  
Popped element: 10  
1.push 2.pop 3.reverse 0.display -1.Exit  
Enter your choice: 2  
Popped element: 20  
1.push 2.pop 3.reverse 0.display -1.Exit  
Enter your choice: 0  
30 40  
1.push 2.pop 3.reverse 0.display -1.Exit  
Enter your choice: -1  
Exiting...  
  
notamitgamer@fedora:~/bsc/semester_2/assignments on ? main X!?  
~ > _
```

Discussion:

- **reverseUsingAdditionalStacks()** empties the original stack into **s1** (reversing the order once), then transfers **s1** into **s2** (reversing it again), then pushes **s2**'s elements back into the original stack, leaving it in reversed order.

Assignment 19

Problem Statement:

WAP to reverse the order of the elements in the stack using additional Queue.

Problem Definition:

Reversing a **Stack** using an auxiliary **Queue** exploits the FIFO order of a queue: elements popped off the stack (in top-to-bottom order) are enqueued, and dequeuing them back (in the same order they were enqueued) and pushing them onto the stack reverses the original arrangement.

Algorithm:

Reverse Stack Using Additional Queue:

```
procedure reverseUsingQueue(S, capacity)
begin
    if(isEmpty(S))
        print "Stack is empty! Nothing to reverse.";
        return;
    end if

    initialize queue Q of capacity;

    while(not isEmpty(S)) do
        val ← pop(S);
        enqueue(Q, val);
    end while

    while(not isEmpty(Q)) do
        val ← dequeue(Q);
        push(S, val);
    end while
end procedure
```

Source Code:

```
#include <iostream>
using namespace std;

class Queue {
private:
    int *arr;
    int front;
    int rear;
    int capacity;

public:
```

```

Queue(int size) {
    capacity = size;
    arr = new int[capacity];
    front = -1;
    rear = -1;
}

bool isEmpty() { return (front == -1 || front > rear); }
bool isFull() { return (rear == capacity - 1); }

void enqueue(int val) {
    if (isFull()) return;
    if (front == -1) front = 0;
    arr[++rear] = val;
}

int dequeue() {
    if (isEmpty()) return -1;
    int val = arr[front++];
    if (front > rear) { front = -1; rear = -1; }
    return val;
}

~Queue() { delete[] arr; }
};

class Stack {
private:
    int *arr;
    int top;
    int capacity;

public:
    Stack(int size) {
        capacity = size;
        arr = new int[capacity];
        top = -1;
    }

    bool isFull() { return (top == capacity - 1); }
    bool isEmpty() { return (top == -1); }

    void push(int val) {
        if (isFull()) { cout << "Stack Overflow!\n"; return; }
        arr[++top] = val;
    }

    int pop() {

```

```

        if (isEmpty()) { cout << "Stack Underflow!\n"; return -1; }
        return arr[top--];
    }

    void display() {
        if (isEmpty()) { cout << "Stack is empty!\n"; return; }
        for (int i = top; i >= 0; i--) cout << arr[i] << " ";
        cout << endl;
    }

    void reverseUsingQueue() {
        if (isEmpty()) { cout << "Stack is empty! Nothing to reverse.\n"; return; }

        Queue q(capacity);
        while (!isEmpty()) q.enqueue(pop());
        while (!q.isEmpty()) push(q.dequeue());
    }

    ~Stack() { delete[] arr; }
};

int main() {
    int size, choice, val;
    cout << "Enter the capacity of the stack: ";
    cin >> size;
    if (size <= 0) { cout << "Invalid stack size!\n"; return 0; }

    Stack s(size);

    while (true) {
        cout << "1.push 2.pop 3.reverse 0.display -1.Exit\n";
        cout << "Enter your choice: ";
        cin >> choice;

        switch (choice) {
            case 1:
                cout << "Enter value: ";
                cin >> val;
                s.push(val);
                break;
            case 2: {
                int v = s.pop();
                if (v != -1) cout << "Popped element: " << v << endl;
                break;
            }
            case 3:
                s.reverseUsingQueue();
                break;
        }
    }
}

```

```

        case 0:
            s.display();
            break;
        case -1:
            cout << "Exiting...\n";
            return 0;
        default:
            cout << "Wrong choice.\n";
    }
}
}

```

Output:

```

notamitgamer@fedora:~/bsc/semester_2/assignments on 7 main X!?
~ ) g++ assignment_18.cpp

notamitgamer@fedora:~/bsc/semester_2/assignments on 7 main X!?
~ ) ./a.out
Enter the capacity of the stack: 4
1.push 2.pop 3.reverse 0.display -1.Exit
Enter your choice: 1
Enter value: 10
1.push 2.pop 3.reverse 0.display -1.Exit
Enter your choice: 1
Enter value: 20
1.push 2.pop 3.reverse 0.display -1.Exit
Enter your choice: 1
Enter value: 30
1.push 2.pop 3.reverse 0.display -1.Exit
Enter your choice: 1
Enter value: 40
1.push 2.pop 3.reverse 0.display -1.Exit
Enter your choice: 1
Enter value: 50
Stack Overflow!
1.push 2.pop 3.reverse 0.display -1.Exit
Enter your choice: 0
40 30 20 10
1.push 2.pop 3.reverse 0.display -1.Exit
Enter your choice: 3
1.push 2.pop 3.reverse 0.display -1.Exit
Enter your choice: 0
10 20 30 40
1.push 2.pop 3.reverse 0.display -1.Exit
Enter your choice: 2
Popped element: 10
1.push 2.pop 3.reverse 0.display -1.Exit
Enter your choice: 2
Popped element: 20
1.push 2.pop 3.reverse 0.display -1.Exit
Enter your choice: 0
30 40
1.push 2.pop 3.reverse 0.display -1.Exit
Enter your choice: -1
Exiting...

notamitgamer@fedora:~/bsc/semester_2/assignments on 7 main X!?
~ ) -

```

Discussion:

- **reverseUsingQueue()** dequeues elements from the stack and enqueues them into a queue in top-to-bottom order, then dequeues them from the queue (same order) and pushes them back onto the stack, producing the reversed arrangement.